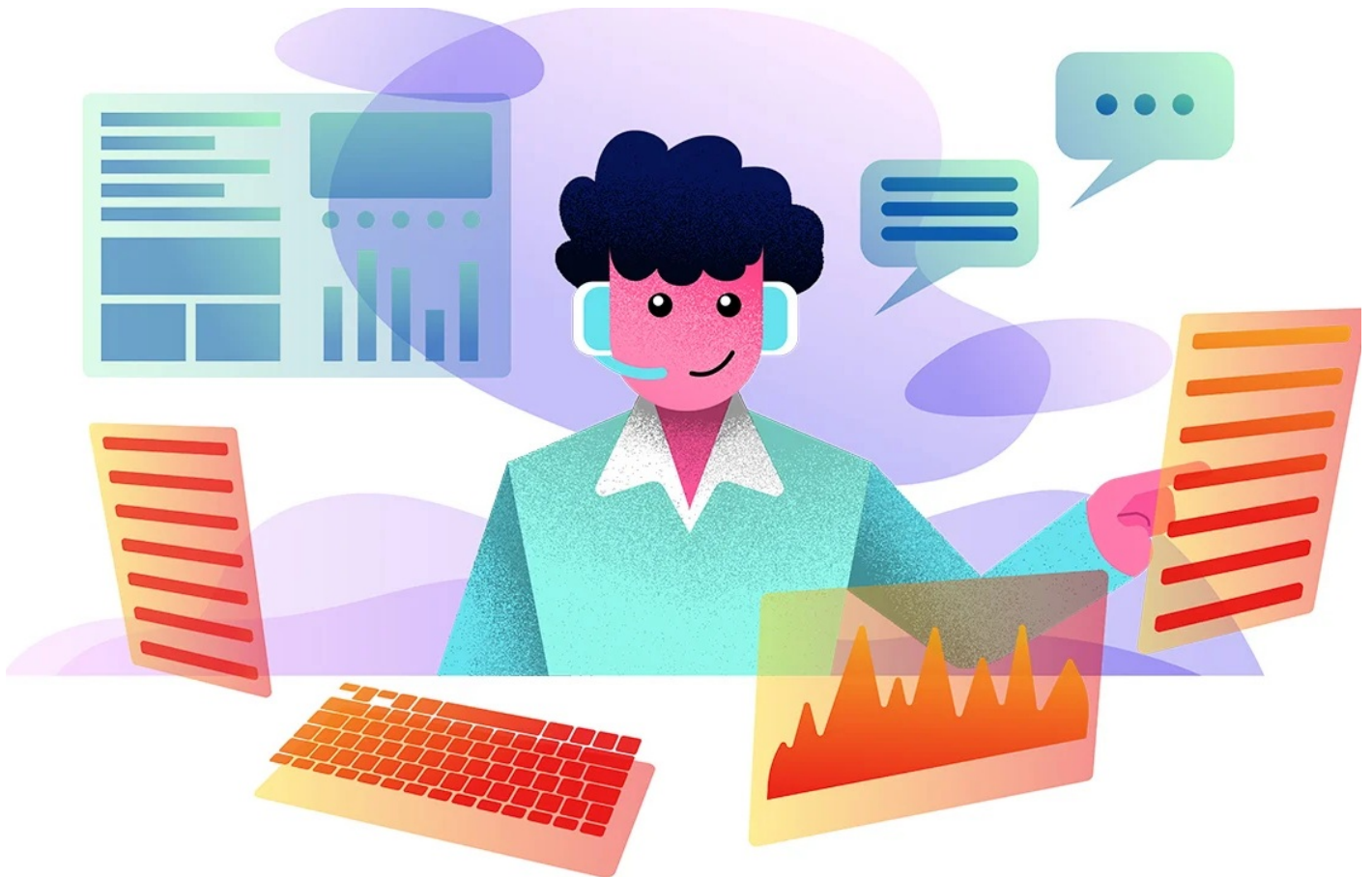


PL/SQL & SQL Coding Guidelines



Tips for Development & Operation

Document Version 4.0
© 2021 Trivadis AG

trivadis

Table of Contents

Table of Contents	2
About	6
Foreword	6
License	7
Trademarks	7
Disclaimer	7
Revision History	7
Introduction	8
Scope	8
Document Conventions	8
SQALE characteristics and subcharacteristics	8
Severity of the rule	10
Keywords used	11
Validator support	11
Why are standards important	11
We have other standards	11
We do not agree with all your standards	12
Naming Conventions	13
General Guidelines	13
Naming Conventions for PL/SQL	13
Database Object Naming Conventions	14
Collection Type	14
Column	15
Check Constraint	15
DML / Instead of Trigger	15
Foreign Key Constraint	15
Function	16
Index	16
Object Type	16
Package	16
Primary Key Constraint	16
Procedure	17
Sequence	17
Synonym	17
System Trigger	17
Table	18
Temporary Table (Global Temporary Table)	18
Unique Key Constraint	18
View	19
Coding Style	20
Formatting	20
Rules	20
Example	20
Code Commenting	22
Conventions	22
Commenting Tags	22
Example	22
Language Usage	23
General	23
G-1010: Try to label your sub blocks.	23
G-1020: Always have a matching loop or block label.	24
G-1030: Avoid defining variables that are not used.	26
G-1040: Avoid dead code.	27

G-1050: Avoid using literals in your code.	29
G-1060: Avoid storing ROWIDs or UROWIDs in database tables.	31
G-1070: Avoid nesting comment blocks.	32
G-1080: Avoid using the same expression on both sides of a relational comparison operator or a logical operator.	33
Variables & Types	34
General	34
G-2110: Try to use anchored declarations for variables, constants and types.	34
G-2120: Try to have a single location to define your types.	35
G-2130: Try to use subtypes for constructs used often in your code.	36
G-2135: Avoid assigning values to local variables that are not used by a subsequent statement.	37
G-2140: Never initialize variables with NULL.	38
G-2145: Never self-assign a variable.	39
G-2150: Avoid comparisons with NULL value, consider using IS [NOT] NULL.	40
G-2160: Avoid initializing variables using functions in the declaration section.	41
G-2170: Never overload variables.	42
G-2180: Never use quoted identifiers.	43
G-2185: Avoid using overly short names for explicitly or implicitly declared identifiers.	44
G-2190: Avoid using ROWID or UROWID.	45
Numeric Data Types	46
G-2210: Avoid declaring NUMBER variables, constants or subtypes with no precision.	46
G-2220: Try to use PLS_INTEGER instead of NUMBER for arithmetic operations with integer values.	47
G-2230: Try to use SIMPLE_INTEGER datatype when appropriate.	48
Character Data Types	49
G-2310: Avoid using CHAR data type.	49
G-2320: Never use VARCHAR data type.	50
G-2330: Never use zero-length strings to substitute NULL.	51
G-2340: Always define your VARCHAR2 variables using CHAR SEMANTIC (if not defined anchored).	52
Boolean Data Types	53
G-2410: Try to use boolean data type for values with dual meaning.	53
Large Objects	54
G-2510: Avoid using the LONG and LONG RAW data types.	54
Cursor Variables	55
G-2610: Never use self-defined weak ref cursor types.	55
DML & SQL	56
General	56
G-3110: Always specify the target columns when coding an insert statement.	56
G-3115: Avoid self-assigning a column.	57
G-3120: Always use table aliases when your SQL statement involves more than one source.	58
G-3130: Try to use ANSI SQL-92 join syntax.	60
G-3140: Try to use anchored records as targets for your cursors.	61
G-3150: Try to use identity columns for surrogate keys.	62
G-3160: Avoid visible virtual columns.	63
G-3170: Always use DEFAULT ON NULL declarations to assign default values to table columns if you refuse to store NULL values.	65
G-3180: Always specify column names instead of positional references in ORDER BY clauses.	67
G-3185: Never use ROWNUM at the same query level as ORDER BY.	68
G-3190: Avoid using NATURAL JOIN.	69
G-3195: Always use wildcards in a LIKE clause.	70
Bulk Operations	71
G-3210: Always use BULK OPERATIONS (BULK COLLECT, FORALL) whenever you have to execute a DML statement for more than 4 times.	71
Transaction Control	72
G-3310: Never commit within a cursor loop.	72
G-3320: Try to move transactions within a non-cursor loop into procedures.	74
Control Structures	75
CURSOR	75
G-4110: Always use %NOTFOUND instead of NOT %FOUND to check whether a cursor returned data.	75
G-4120: Avoid using %NOTFOUND directly after the FETCH when working with BULK OPERATIONS and LIMIT clause.	76
G-4130: Always close locally opened cursors.	78
G-4140: Avoid executing any statements between a SQL operation and the usage of an implicit cursor attribute.	79
CASE / IF / DECODE / NVL / NVL2 / COALESCE	81
G-4210: Try to use CASE rather than an IF statement with multiple ELSIF paths.	81
G-4220: Try to use CASE rather than DECODE.	82
G-4230: Always use a COALESCE instead of a NVL command, if parameter 2 of the NVL function is a function call or a SELECT statement.	83
G-4240: Always use a CASE instead of a NVL2 command if parameter 2 or 3 of NVL2 is either a function call or a SELECT statement.	84
G-4250: Avoid using identical conditions in different branches of the same IF or CASE statement.	85
G-4260: Avoid inverting boolean conditions with NOT.	86
G-4270: Avoid comparing boolean values to boolean literals.	87
Flow Control	88
G-4310: Never use GOTO statements in your code.	88
G-4320: Always label your loops.	90
G-4325: Never reuse labels in inner scopes.	92

G-4330: Always use a CURSOR FOR loop to process the complete cursor results unless you are using bulk operations.	93
G-4340: Always use a NUMERIC FOR loop to process a dense array.	94
G-4350: Always use 1 as lower and COUNT() as upper bound when looping through a dense array.	95
G-4360: Always use a WHILE loop to process a loose array.	96
G-4370: Avoid using EXIT to stop loop processing unless you are in a basic loop.	97
G-4375: Always use EXIT WHEN instead of an IF statement to exit from a loop.	99
G-4380: Try to label your EXIT WHEN statements.	100
G-4385: Never use a cursor for loop to check whether a cursor returns data.	102
G-4390: Avoid use of unreferenced FOR loop indexes.	103
G-4395: Avoid hard-coded upper or lower bound values with FOR loops.	104
Exception Handling	105
G-5010: Try to use a error/logging framework for your application.	105
G-5020: Never handle unnamed exceptions using the error number.	106
G-5030: Never assign predefined exception names to user defined exceptions.	107
G-5040: Avoid use of WHEN OTHERS clause in an exception section without any other specific handlers.	109
Example (exception to the rule)	109
G-5050: Avoid use of the RAISE_APPLICATION_ERROR built-in procedure with a hard-coded 20nnn error number or hard-coded message.	110
G-5060: Avoid unhandled exceptions.	111
G-5070: Avoid using Oracle predefined exceptions.	112
G-5080: Always use FORMAT_ERROR_BACKTRACE when using FORMAT_ERROR_STACK or SQLERRM.	113
Dynamic SQL	114
G-6010: Always use a character variable to execute dynamic SQL.	114
G-6020: Try to use output bind arguments in the RETURNING INTO clause of dynamic DML statements rather than the USING clause.	115
Stored Objects	116
General	116
G-7110: Try to use named notation when calling program units.	116
G-7120: Always add the name of the program unit to its end keyword.	117
G-7125: Always use CREATE OR REPLACE instead of CREATE alone.	118
G-7130: Always use parameters or pull in definitions rather than referencing external variables in a local program unit.	119
G-7140: Always ensure that locally defined procedures or functions are referenced.	121
G-7150: Try to remove unused parameters.	122
G-7160: Always explicitly state parameter mode.	123
G-7170: Avoid using an IN OUT parameter as IN or OUT only.	124
Packages	126
G-7210: Try to keep your packages small. Include only few procedures and functions that are used in the same context.	126
G-7220: Always use forward declaration for private functions and procedures.	127
G-7230: Avoid declaring global variables public.	129
G-7250: Never use RETURN in package initialization block.	131
Procedures	132
G-7310: Avoid standalone procedures – put your procedures in packages.	132
G-7320: Avoid using RETURN statements in a PROCEDURE.	133
G-7330: Always assign values to OUT parameters.	134
Functions	135
G-7410: Avoid standalone functions – put your functions in packages.	135
G-7420: Always make the RETURN statement the last statement of your function.	136
G-7430: Try to use no more than one RETURN statement within a function.	137
G-7440: Never use OUT parameters to return values from a function.	138
G-7450: Never return a NULL value from a BOOLEAN function.	139
G-7460: Try to define your packaged/standalone function deterministic if appropriate.	140
Oracle Supplied Packages	141
G-7510: Always prefix ORACLE supplied packages with owner schema name.	141
Object Types	142
Triggers	143
G-7710: Avoid cascading triggers.	143
G-7720: Never use multiple UPDATE OF in trigger event clause.	145
G-7730: Avoid multiple DML events per trigger if primary key is assigned in trigger.	146
Sequences	148
G-7810: Never use SQL inside PL/SQL to read sequence numbers (or SYSDATE).	148
Patterns	149
Checking the Number of Rows	149
G-8110: Never use SELECT COUNT(*) if you are only interested in the existence of a row.	149
G-8120: Never check existence of a row to decide whether to create it or not.	150
Access objects of foreign application schemas	151
G-8210: Always use synonyms when accessing objects of another application schema.	151
Validating input parameter size	152
G-8310: Always validate input parameter size by assigning the parameter to a size limited variable in the declaration section of program unit.	152
Ensure single execution at a time of a program unit	154
G-8410: Always use application locks to ensure a program unit is only running once at a given time.	154
Use dbms_application_info package to follow progress of a process	157

G-8510: Always use dbms_application_info to track program process transiently.	157
Function Usage	158
G-9010: Always use a format model in string to date/time conversion functions.	158
G-9020: Try to use a format model and NLS_NUMERIC_CHARACTERS in string to number conversion functions.	159
G-9030: Try to define a default value on conversion errors.	160
Complexity Analysis	161
Halstead Metrics	161
Calculation	161
McCabe's Cyclomatic Complexity	161
Description	161
Calculation	162
Code Reviews	164
Tool Support	165
Development	165
Setting the preferences	165
Activate PLSQL Cop using context menu	165
Software metrics	166
Appendix	168
A - PL/SQL & SQL Coding Guidelines as PDF	168
B - Mapping new guidelines to prior versions	168

About

Foreword



In the I.T. world of today, robust and secure applications are becoming more and more important. Many business processes no longer work without I.T. and the dependence of businesses on their I.T. has grown tremendously, meaning we need robust and maintainable applications. An important requirement is to have standards and guidelines, which make it possible to maintain source code created by a number of people quickly and easily. This forms the basis of well functioning off- or on-shoring strategy, as it allows quality assurance to be carried out efficiently at the source.

Good standards and guidelines are based on the wealth of experience and knowledge gained from past (and future?) problems, such as those, which can arise in a cloud environment, for example.

A handwritten signature in black ink, reading "h. Lankes".

Urban Lankes
Chairman of the Board of Directors
Trivadis



The Oracle Database Developer community is made stronger by resources freely shared by experts around the world, such as the Trivadis Coding Guidelines. If you have not yet adopted standards for writing SQL and PL/SQL in your applications, this is a great place to start.

A handwritten signature in black ink, reading "Steven Feuerstein".

Steven Feuerstein
Senior Advisor
Insum Solutions



Coding Guidelines are a crucial part of software development. It is a matter of fact, that code is more often read than written – therefore we should take efforts to ease the work of the reader, which is not necessarily the author.

I am convinced that this standard may be a good starting point for your own guidelines.

License

The Trivadis PL/SQL & SQL Coding Guidelines are licensed under the Apache License, Version 2.0. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>.

Trademarks

All terms that are known trademarks or service marks have been capitalized. All trademarks are the property of their respective owners.

Disclaimer

The authors and publisher shall have neither liability nor responsibility to any person or entity with respect to the loss or damages arising from the information contained in this work. This work may include inaccuracies or typographical errors and solely represent the opinions of the authors. Changes are periodically made to this document without notice. The authors reserve the right to revise this document at any time without notice.

Revision History

The first version of these guidelines was compiled by Roger Troller on March 17, 2009. Jörn Kulesa, Daniela Reiner, Richard Bushnell, Andreas Flubacher and Thomas Mauch helped Roger complete version 1.2 until August 21, 2009. This was the first GA version. The handy printed version in A5 format was distributed free of charge at the DOAG Annual Conference and on other occasions. Since then Roger updated the guidelines regularly. Philipp Salvisberg was involved in the review process for version 3.0 which was a major update. Philipp took the lead, after Roger left Trivadis in 2016. In 2020 Kim Berg Hansen started handling guidelines maintenance, letting Philipp concentrate on the related [Trivadis PL/SQL Cop](#) tool.

Since July, 7 2018 these guidelines are hosted on GitHub. Ready to be enhanced by the community and forked to fit specific needs.

On <https://github.com/Trivadis/plsql-and-sql-coding-guidelines/releases> you find the release information for every version since 1.2.

Introduction

This document describes rules and recommendations for developing applications using the PL/SQL & SQL Language.

Scope

This document applies to the PL/SQL and SQL language as used within ORACLE databases and tools, which access ORACLE databases.

Document Conventions

SQALE (Software Quality Assessment based on Lifecycle Expectations) is a method to support the evaluation of a software application source code. It is a generic method, independent of the language and source code analysis tools.

SQALE characteristics and subcharacteristics

Characteristic	Description and Subcharacteristics
Changeability	The capability of the software product to enable a specified modification to be implemented. • Architecture related changeability • Logic related changeability • Data related changeability
Efficiency	The capability of the software product to provide appropriate performance, relative to the amount of resources used, under stated conditions. • Memory use • Processor use • Network use
Maintainability	The capability of the software product to be modified. Modifications may include corrections, improvements or adaptation of the software to changes in environment, and in requirements and functional specifications. • Understandability • Readability
Portability	The capability of the software product to be transferred from one environment to another. • Compiler related portability • Hardware related portability • Language related portability • OS related portability • Software related portability • Time zone related portability.

Reliability	The capability of the software product to maintain a specified level of performance when used under specified conditions. • Architecture related reliability • Data related reliability • Exception handling • Fault tolerance • Instruction related reliability • Logic related reliability • Resource related reliability • Synchronization related reliability • Unit tests coverage.
Reusability	The capability of the software product to be reused within the development process. • Modularity • Transportability.
Security	The capability of the software product to protect information and data so that unauthorized persons or systems cannot read or modify them and authorized persons or systems are not denied access to them. • API abuse • Errors (e.g. leaving a system in a vulnerable state) • Input validation and representation • Security features.
Testability	The capability of the software product to enable modified software to be validated. • Integration level testability • Unit level testability.

Severity of the rule

Blocker

Will or may result in a bug.

Critical

Will have a high/direct impact on the maintenance cost.

Major

Will have a medium/potential impact on the maintenance cost.

Minor

Will have a low impact on the maintenance cost.

Info

Very low impact; it is just a remediation cost report.

Keywords used

Keyword	Meaning
Always	Emphasizes this rule must be enforced.
Never	Emphasizes this action must not happen.
Avoid	Emphasizes that the action should be prevented, but some exceptions may exist.
Try	Emphasizes that the rule should be attempted whenever possible and appropriate.
Example	Precedes text used to illustrate a rule or a recommendation.
Reason	Explains the thoughts and purpose behind a rule or a recommendation.
Restriction	Describes the circumstances to be fulfilled to make use of a rule.

Validator support

The tool PL/SQL Cop (see the "Tool Support" chapter) cannot support *all* the guidelines in this document. Those guidelines that are *not* supported by PL/SQL Cop validators are marked like this:

 **Unsupported in PL/SQL Cop Validators**

Reason why the specific guideline is not supported by the validators.

The PL/SQL Cop repository documents the [details of validator limitations](#).

Why are standards important

For a machine executing a program, code formatting is of no importance. However, for the human eye, well-formatted code is much easier to read. Modern tools can help to implement format and coding rules.

Implementing formatting and coding standards has the following advantages for PL/SQL development:

- Well-formatted code is easier to read, analyze and maintain (not only for the author but also for other developers).
- The developers do not have to define their own guidelines - it is already defined.
- The code has a structure that makes it easier to avoid making errors.
- The code is more efficient concerning performance and organization of the whole application.
- The code is more modular and thus easier to use for other applications.

We have other standards

This document only defines possible standards. These standards are not written in stone, but are meant as guidelines. If standards already exist, and they are different from those in this document, it makes no sense to change them.

We do not agree with all your standards

There are basically two types of standards.

1. Non-controversial

These standards make sense. There is no reason not to follow them. An example of this category is [G-2150](#): Avoid comparisons with NULL value, consider using IS [NOT] NULL.

2. Controversial

Almost every rule/guideline falls into this category. An example of this category is [3 space indentation](#). - Why not 2 or 4 or even 8? Why not use tabs? You can argue in favor of all these options. In most cases it does not really matter which option you choose. Being consistent is more important. In this case it will make the code easier to read.

For very controversial rules, we have started to include the reasoning either as a footnote or directly in the text.

Usually it is not helpful to open an issue on GitHub to request to change a highly controversial rule such as the one mentioned. For example, use 2 spaces instead of 3 spaces for an indentation. This leads to a discussion where the people in favor of 4 spaces start to argue as well. There is no right or wrong here. You just have to agree on a standard.

More effective is to fork [this repository](#) and amend the standards to fit your needs/expectations.

Naming Conventions

General Guidelines

1. Never use names with a leading numeric character.
2. Always choose meaningful and specific names.
3. Avoid using abbreviations unless the full name is excessively long.
4. Avoid long abbreviations. Abbreviations should be shorter than 5 characters.
5. Any abbreviations must be widely known and accepted.
6. Create a glossary with all accepted abbreviations.
7. Never use ORACLE keywords as names. A list of ORACLEs keywords may be found in the dictionary view `v$reserved_words`.
8. Avoid adding redundant or meaningless prefixes and suffixes to identifiers.
Example: `create table emp_table`.
9. Always use one spoken language (e.g. English, German, French) for all objects in your application.
10. Always use the same names for elements with the same meaning.

Naming Conventions for PL/SQL

In general, ORACLE is not case sensitive with names. A variable named `personname` is equal to one named `PersonName`, as well as to one named `PERSONNAME`. Some products (e.g. TMDA by Trivadis, APEX, OWB) put each name within double quotes (") so ORACLE will treat these names to be case sensitive. Using case sensitive variable names force developers to use double quotes for each reference to the variable. Our recommendation is to write all names in lowercase and to avoid double quoted identifiers.

A widely used convention is to follow a `{prefix}variablecontent{suffix}` pattern.

The following table shows a possible set of naming conventions.

Identifier	Prefix	Suffix	Example
Global Variable	<code>g_</code>		<code>g_version</code>
Local Variable	<code>l_</code>		<code>l_version</code>
Cursor	<code>c_</code>		<code>c_employees</code>
Record	<code>r_</code>		<code>r_employee</code>
Array / Table	<code>t_</code>		<code>t_employees</code>
Object	<code>o_</code>		<code>o_employee</code>
Cursor Parameter	<code>p_</code>		<code>p_empno</code>

In Parameter	<code>in_</code>		<code>in_empno</code>
Out Parameter	<code>out_</code>		<code>out_ename</code>
In/Out Parameter	<code>io_</code>		<code>io_employee</code>
Record Type Definitions	<code>r_</code>	<code>_type</code>	<code>r_employee_type</code>
Array/Table Type Definitions	<code>t_</code>	<code>_type</code>	<code>t_employees_type</code>
Exception	<code>e_</code>		<code>e_employee_exists</code>
Constants	<code>co_</code>		<code>co_empno</code>
Subtypes		<code>_type</code>	<code>big_string_type</code>

Database Object Naming Conventions

Never enclose object names (table names, column names, etc.) in double quotes to enforce mixed case or lower case object names in the data dictionary.

Collection Type

A collection type should include the name of the collected objects in their name. Furthermore, they should have the suffix `_ct` to identify it as a collection.

Optionally prefixed by a project abbreviation.

Examples:

- `employees_ct`
- `orders_ct`

Column

Singular name of what is stored in the column (unless the column data type is a collection, in this case you use plural names)

Add a comment to the database dictionary for every column.

Check Constraint

Table name or table abbreviation followed by the column and/or role of the check constraint, a `_ck` and an optional number suffix.

Examples:

- `employees_salary_min_ck`
- `orders_mode_ck`

DML / Instead of Trigger

Choose a naming convention that includes:

either

- the name of the object the trigger is added to,
- any of the triggering events:
 - `_br_iud` for Before Row on Insert, Update and Delete
 - `_io_id` for Instead of Insert and Delete

or

- the name of the object the trigger is added to,
- the activity done by the trigger,
- the suffix `_trg`

Examples:

- `employees_br_iud`
- `orders_audit_trg`
- `orders_journal_trg`

Foreign Key Constraint

Table abbreviation followed by referenced table abbreviation followed by a `_fk` and an optional number suffix.

Examples:

- `empl_dept_fk`
- `sct_icmd_ic_fk1`

Function

Name is built from a verb followed by a noun in general. Nevertheless, it is not sensible to call a function `get_...` as a function always gets something.

The name of the function should answer the question "What is the outcome of the function?"

Optionally prefixed by a project abbreviation.

Example: `employee_by_id`

If more than one function provides the same outcome, you have to be more specific with the name.

Index

Indexes serving a constraint (primary, unique or foreign key) are named accordingly.

Other indexes should have the name of the table and columns (or their purpose) in their name and should also have `_idx` as a suffix.

Object Type

The name of an object type is built by its content (singular) followed by a `_ot` suffix.

Optionally prefixed by a project abbreviation.

Example: `employee_ot`

Package

Name is built from the content that is contained within the package.

Optionally prefixed by a project abbreviation.

Examples:

- `employees_api` - API for the employee table
- `logging_up` - Utilities including logging support

Primary Key Constraint

Table name or table abbreviation followed by the suffix `_pk`.

Examples:

- `employees_pk`
- `departments_pk`
- `sct_contracts_pk`

Procedure

Name is built from a verb followed by a noun. The name of the procedure should answer the question “What is done?”

Procedures and functions are often named with underscores between words because some editors write all letters in uppercase in the object tree, so it is difficult to read them.

Optionally prefixed by a project abbreviation.

Examples:

- `calculate_salary`
- `set_hiredate`
- `check_order_state`

Sequence

Name is built from the table name (or its abbreviation) the sequence serves as primary key generator and the suffix `_seq` or the purpose of the sequence followed by a `_seq`.

Optionally prefixed by a project abbreviation.

Examples:

- `employees_seq`
- `order_number_seq`

Synonym

Synonyms should be used to address an object in a foreign schema rather than to rename an object. Therefore, synonyms should share the name with the referenced object.

System Trigger

Name of the event the trigger is based on.

- Activity done by the trigger
- Suffix `_trg`

Examples:

- `ddl_audit_trg`
- `logon_trg`

Table

Plural¹ name of what is contained in the table (unless the table is designed to always hold one row only – then you should use a singular name).

Suffixed by `_eb` when protected by an editioning view.

Add a comment to the database dictionary for every table and every column in the table.

Optionally prefixed by a project abbreviation.

Examples:

- `employees`
- `departments`
- `countries_eb` - table interfaced by an editioning view named `countries`
- `sct_contracts`
- `sct_contract_lines`
- `sct_incentive_modules`

Temporary Table (Global Temporary Table)

Naming as described for tables.

Optionally suffixed by `_tmp`

Optionally prefixed by a project abbreviation.

Examples:

- `employees_tmp`
- `contracts_tmp`

Unique Key Constraint

Table name or table abbreviation followed by the role of the unique key constraint, a `_uk` and an optional number suffix.

Examples:

- `employees_name_uk`
- `departments_deptno_uk`
- `sct_contracts_uk`
- `sct_coli_uk`
- `sct_icmd_uk1`

View

Plural¹ name of what is contained in the view. Optionally suffixed by an indicator identifying the object as a view (mostly used, when a 1:1 view layer lies above the table layer)

Editing views are named like the original underlying table to avoid changing the existing application code when introducing edition based redefinition (EBR).

Add a comment to the database dictionary for every view and every column.

Optionally prefixed by a project abbreviation.

Examples:

- `active_orders`
- `orders_v` - a view to the orders table
- `countries` - an editing view for table `countries_eb`

Coding Style

Formatting

Rules

Rule	Description
1	Keywords and names are written in lowercase ² .
2	3 space indentation ³ .
3	One command per line.
4	Keywords <code>loop</code> , <code>else</code> , <code>elsif</code> , <code>end if</code> , <code>when</code> on a new line.
5	Commas in front of separated elements.
6	Call parameters aligned, operators aligned, values aligned.
7	SQL keywords are right aligned within a SQL command.
8	Within a program unit only line comments <code>--</code> are used.
9	Brackets are used when needed or when helpful to clarify a construct.

Example

```

1  procedure set_salary(in_employee_id in employees.employee_id%type) is
2      cursor c_employees(p_employee_id in employees.employee_id%type) is
3          select last_name
4                 ,first_name
5                 ,salary
6          from employees
7          where employee_id = p_employee_id
8          order by last_name
9                 ,first_name;
10
11      r_employee      c_employees%rowtype;
12      l_new_salary    employees.salary%type;
13  begin
14      open  c_employees(p_employee_id => in_employee_id);
15      fetch c_employees into r_employee;
16      close c_employees;
17
18      new_salary (in_employee_id => in_employee_id
19                  ,out_salary      => l_new_salary);
20
21      -- Check whether salary has changed
22      if r_employee.salary <> l_new_salary then
23          update employees
24              set salary = l_new_salary
25              where employee_id = in_employee_id;
26      end if;
27  end set_salary;

```

Code Commenting

Conventions

Inside a program unit only use the line commenting technique `--` unless you temporarily deactivate code sections for testing.

To comment the source code for later document generation, comments like `/** ... */` are used. Within these documentation comments, tags may be used to define the documentation structure.

Tools like ORACLE SQL Developer or PL/SQL Developer include documentation functionality based on a javadoc-like tagging.

Commenting Tags

Tag	Meaning	Example
<code>param</code>	Description of a parameter.	<code>@param in_string input string</code>
<code>return</code>	Description of the return value of a function.	<code>@return result of the calculation</code>
<code>throws</code>	Describe errors that may be raised by the program unit.	<code>@throws NO_DATA_FOUND</code>

Example

This is an example using the documentation capabilities of SQL Developer.

```
1  /**
2  Check whether we passed a valid sql name
3
4  @param  in_name  string to be checked
5  @return  in_name if the string represents a valid sql name
6  @throws  ORA-44003: invalid SQL name
7
8  <b>Call Example:</b>
9  <pre>
10      select TVDAssert.valid_sql_name('TEST') from dual;
11      select TVDAssert.valid_sql_name('123') from dual
12  </pre>
13  */
```

Language Usage

General

G-1010: Try to label your sub blocks.

Minor

Maintainability

Reason

It's a good alternative for comments to indicate the start and end of a named processing.

Example (bad)

```
1  begin
2      begin
3          null;
4      end;
5
6      begin
7          null;
8      end;
9  end;
10 /
```

Example (good)

```
1  begin
2      <<prepare_data>>
3      begin
4          null;
5      end prepare_data;
6
7      <<process_data>>
8      begin
9          null;
10     end process_data;
11 end good;
12 /
```

G-1020: Always have a matching loop or block label.

Minor

Maintainability

Reason

Use a label directly in front of loops and nested anonymous blocks:

- To give a name to that portion of code and thereby self-document what it is doing.
- So that you can repeat that name with the `end` statement of that block or loop.

Example (bad)

```
1  declare
2      i integer;
3      co_min_value constant integer := 1;
4      co_max_value constant integer := 10;
5      co_increment constant integer := 1;
6  begin
7      <<prepare_data>>
8      begin
9          null;
10     end;
11
12     <<process_data>>
13     begin
14         null;
15     end;
16
17     i := co_min_value;
18     <<while_loop>>
19     while (i <= co_max_value)
20     loop
21         i := i + co_increment;
22     end loop;
23
24     <<basic_loop>>
25     loop
26         exit basic_loop;
27     end loop;
28
29     <<for_loop>>
30     for i in co_min_value..co_max_value
31     loop
32         sys.dbms_output.put_line(i);
33     end loop;
34 end;
35 /
```

Example (good)


```

1  declare
2      i integer;
3      co_min_value constant integer := 1;
4      co_max_value constant integer := 10;
5      co_increment constant integer := 1;
6  begin
7      <<prepare_data>>
8      begin
9          null;
10     end prepare_data;
11
12     <<process_data>>
13     begin
14         null;
15     end process_data;
16
17     i := co_min_value;
18     <<while_loop>>
19     while (i <= co_max_value)
20     loop
21         i := i + co_increment;
22     end loop while_loop;
23
24     <<basic_loop>>
25     loop
26         exit basic_loop;
27     end loop basic_loop;
28
29     <<for_loop>>
30     for i in co_min_value..co_max_value
31     loop
32         sys.dbms_output.put_line(i);
33     end loop for_loop;
34 end;
35 /

```

G-1030: Avoid defining variables that are not used.

Minor

Efficiency, Maintainability

Reason

Unused variables decrease the maintainability and readability of your code.

Example (bad)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          l_last_name employees.last_name%type;
4          l_first_name employees.first_name%type;
5          co_department_id constant departments.department_id%type := 10;
6          e_good exception;
7      begin
8          select e.last_name
9              into l_last_name
10             from employees e
11             where e.department_id = co_department_id;
12      exception
13          when no_data_found then null; -- handle_no_data_found;
14          when too_many_rows then null; -- handle_too_many_rows;
15      end my_proc;
16  end my_package;
17  /
```

Example (good)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          l_last_name employees.last_name%type;
4          co_department_id constant departments.department_id%type := 10;
5          e_good exception;
6      begin
7          select e.last_name
8              into l_last_name
9             from employees e
10             where e.department_id = co_department_id;
11
12          raise e_good;
13      exception
14          when no_data_found then null; -- handle_no_data_found;
15          when too_many_rows then null; -- handle_too_many_rows;
16      end my_proc;
17  end my_package;
18  /
```

G-1040: Avoid dead code.

Minor

Maintainability

Reason

Any part of your code, which is no longer used or cannot be reached, should be eliminated from your programs to simplify the code.

Example (bad)

```
1  declare
2      co_dept_purchasing constant departments.department_id%type := 30;
3  begin
4      if 2=3 then
5          null; -- some dead code here
6      end if;
7
8      null; -- some enabled code here
9
10     <<my_loop>>
11     loop
12         exit my_loop;
13         null; -- some dead code here
14     end loop my_loop;
15
16     null; -- some other enabled code here
17
18     case
19         when 1 = 1 and 'x' = 'y' then
20             null; -- some dead code here
21         else
22             null; -- some further enabled code here
23     end case;
24
25     <<my_loop2>>
26     for r_emp in (select last_name
27                   from employees
28                   where department_id = co_dept_purchasing
29                      or commission_pct is not null
30                      and 5=6)
31         -- "or commission_pct is not null" is dead code
32     loop
33         sys.dbms_output.put_line(r_emp.last_name);
34     end loop my_loop2;
35
36     return;
37     null; -- some dead code here
38 end;
39 /
```

Example (good)

```

1  declare
2      co_dept_admin constant dept.deptno%type := 10;
3  begin
4      null; -- some enabled code here
5      null; -- some other enabled code here
6      null; -- some further enabled code here
7
8      <<my_loop2>>
9      for r_emp in (select last_name
10                     from employees
11                     where department_id = co_dept_admin
12                           or commission_pct is not null)
13      loop
14          sys.dbms_output.put_line(r_emp.last_name);
15      end loop my_loop2;
16  end;
17  /

```

G-1050: Avoid using literals in your code.

Minor

Changeability

Reason

Literals are often used more than once in your code. Having them defined as a constant reduces typos in your code and improves the maintainability.

All constants should be collated in just one package used as a library. If these constants should be used in SQL too it is good practice to write a deterministic package function for every constant.

In specific situations this rule could lead to an extreme plethora of constants, for example if you use Logger like `logger.append_param(p_params => l_params, p_name => 'p_param1_todo', p_val => p_param1_todo);`, where the value for `p_name` always should be the name of the variable that is passed to `p_val`. For such cases it would be overkill to add constants for every single variable name you are logging, so if you use Logger or similar, consider making that an exception to the rule, just document exactly which exceptions you will allow and stick to them.

Example (bad)

```
1  declare
2      l_job employees.job_id%type;
3  begin
4      select e.job_id
5          into l_job
6          from employees e
7          where e.manager_id is null;
8
9      if l_job = 'AD_PRES' then
10         null;
11     end if;
12 exception
13     when no_data_found then
14         null; -- handle_no_data_found;
15     when too_many_rows then
16         null; -- handle_too_many_rows;
17 end;
18 /
```

Example (good)

```

1  create or replace package constants_up is
2      co_president constant employees.job_id%type := 'AD_PRES';
3  end constants_up;
4  /
5
6  declare
7      l_job employees.job_id%type;
8  begin
9      select e.job_id
10         into l_job
11         from employees e
12         where e.manager_id is null;
13
14     if l_job = constants_up.co_president then
15         null;
16     end if;
17 exception
18     when no_data_found then
19         null; -- handle_no_data_found;
20     when too_many_rows then
21         null; -- handle_too_many_rows;
22 end;
23 /

```

G-1060: Avoid storing ROWIDs or UROWIDs in database tables.

Major

Reliability

Reason

It is an extremely dangerous practice to store `rowid`'s in a table, except for some very limited scenarios of runtime duration. Any manually explicit or system generated implicit table reorganization will reassign the row's `rowid` and break the data consistency.

Instead of using `rowid` for later reference to the original row one should use the primary key column(s).

Example (bad)

```
1  begin
2      insert into employees_log (employee_id
3                                ,last_name
4                                ,first_name
5                                ,rid)
6      select employee_id
7             ,last_name
8             ,first_name
9             ,rowid
10     from employees;
11 end;
12 /
```

Example (good)

```
1  begin
2      insert into employees_log (employee_id
3                                ,last_name
4                                ,first_name)
5      select employee_id
6             ,last_name
7             ,first_name
8      from employees;
9  end;
10 /
```

G-1070: Avoid nesting comment blocks.

Minor

Maintainability

Reason

Having an end-of-comment within a block comment will end that block-comment. This does not only influence your code but is also very hard to read.

Example (bad)

```
1  begin
2      /* comment one -- nested comment two */
3      null;
4      -- comment three /* nested comment four */
5      null;
6  end;
7  /
```

Example (good)

```
1  begin
2      /* comment one, comment two */
3      null;
4      -- comment three, comment four
5      null;
6  end;
7  /
```


G-1080: Avoid using the same expression on both sides of a relational comparison operator or a logical operator.

Minor

Maintainability, Efficiency, Testability

Reason

Using the same value on either side of a binary operator is almost always a mistake. In the case of logical operators, it is either a copy/paste error and therefore a bug, or it is simply wasted code and should be simplified.

This rule ignores operators `+`, `*` and `||`, and expressions: `1=1`, `1<>1`, `1!=1`, `1~=1` and `1^=1`.

Example (bad)

```
1  select emp.first_name
2         , emp.last_name
3         , emp.salary
4         , emp.hire_date
5  from employees emp
6  where emp.salary > 3000 or emp.salary > 3000
7  order by emp.last_name, emp.first_name;
```

Example (good)

```
1  select emp.first_name
2         , emp.last_name
3         , emp.salary
4         , emp.hire_date
5  from employees emp
6  where emp.salary > 3000
7  order by emp.last_name, emp.first_name;
```

Variables & Types

General

G-2110: Try to use anchored declarations for variables, constants and types.

Major

Maintainability, Reliability

REASON

Changing the size of the database column `last_name` in the `employees` table from `varchar2(20)` to `varchar2(30)` will result in an error within your code whenever a value larger than the hard coded size is read from the table. This can be avoided using anchored declarations.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          l_last_name  varchar2(20 char);
4          co_first_row constant integer := 1;
5      begin
6          select e.last_name
7              into l_last_name
8              from employees e
9              where rownum = co_first_row;
10     exception
11         when no_data_found then null; -- handle no_data_found
12         when too_many_rows then null; -- handle too_many_rows (impossible)
13     end my_proc;
14 end my_package;
15 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          l_last_name  employees.last_name%type;
4          co_first_row constant integer := 1;
5      begin
6          select e.last_name
7              into l_last_name
8              from employees e
9              where rownum = co_first_row;
10     exception
11         when no_data_found then null; -- handle no_data_found
12         when too_many_rows then null; -- handle too_many_rows (impossible)
13     end my_proc;
14 end my_package;
15 /
```

G-2120: Try to have a single location to define your types.

Minor

Changeability

REASON

Single point of change when changing the data type. No need to argue where to define types or where to look for existing definitions.

A single location could be either a type specification package or the database (database-defined types).

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          subtype big_string_type is varchar2(1000 char);
4          l_note big_string_type;
5      begin
6          l_note := some_function();
7      end my_proc;
8  end my_package;
9  /
```

EXAMPLE (GOOD)

```
1  create or replace package types_up is
2      subtype big_string_type is varchar2(1000 char);
3  end types_up;
4  /
5
6  create or replace package body my_package is
7      procedure my_proc is
8          l_note types_up.big_string_type;
9      begin
10         l_note := some_function();
11     end my_proc;
12 end my_package;
13 /
```

G-2130: Try to use subtypes for constructs used often in your code.

Minor

Changeability

REASON

Single point of change when changing the data type.

Your code will be easier to read as the usage of a variable/constant may be derived from its definition.

EXAMPLES OF POSSIBLE SUBTYPE DEFINITIONS

Type	Usage
<code>ora_name_type</code>	Object corresponding to the ORACLE naming conventions (table, variable, column, package, etc.).
<code>max_vc2_type</code>	String variable with maximal VARCHAR2 size.
<code>array_index_type</code>	Best fitting data type for array navigation.
<code>id_type</code>	Data type used for all primary key (id) columns.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          l_note varchar2(1000 char);
4      begin
5          l_note := some_function();
6      end my_proc;
7  end my_package;
8  /
```

EXAMPLE (GOOD)

```
1  create or replace package types_up is
2      subtype big_string_type is varchar2(1000 char);
3  end types_up;
4  /
5
6  create or replace package body my_package is
7      procedure my_proc is
8          l_note types_up.big_string_type;
9      begin
10         l_note := some_function();
11     end my_proc;
12 end my_package;
13 /
```

G-2135: Avoid assigning values to local variables that are not used by a subsequent statement.

Major

Efficiency, Maintainability, Testability

REASON

Expending resources calculating and assigning values to a local variable and never use the value subsequently is at best a waste, at worst indicative of a mistake that leads to a bug.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          co_employee_id constant employees.employee_id%type := 1042;
4          l_last_name     employees.last_name%type;
5          l_message       varchar2(100 char);
6      begin
7          select emp.last_name
8              into l_last_name
9              from employees emp
10             where emp.employee_id = co_employee_id;
11
12             l_message := 'Hello, ' || l_last_name;
13      exception
14          when no_data_found then null; -- handle_no_data_found;
15          when too_many_rows then null; -- handle_too_many_rows;
16      end my_proc;
17  end my_package;
18  /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      procedure my_proc is
3          co_employee_id constant employees.employee_id%type := 1042;
4          l_last_name     employees.last_name%type;
5          l_message       varchar2(100 char);
6      begin
7          select emp.last_name
8              into l_last_name
9              from employees emp
10             where emp.employee_id = co_employee_id;
11
12             l_message := 'Hello, ' || l_last_name;
13
14             message_api.send_message(l_message);
15      exception
16          when no_data_found then null; -- handle_no_data_found;
17          when too_many_rows then null; -- handle_too_many_rows;
18      end my_proc;
19  end my_package;
20  /
```

G-2140: Never initialize variables with NULL.

Minor

Maintainability

REASON

Variables are initialized to `null` by default.

EXAMPLE (BAD)

```
1  declare
2      l_note big_string_type := null;
3  begin
4      sys.dbms_output.put_line(l_note);
5  end;
6  /
```

EXAMPLE (GOOD)

```
1  declare
2      l_note big_string_type;
3  begin
4      sys.dbms_output.put_line(l_note);
5  end;
6  /
```

G-2145: Never self-assign a variable.

Minor

Maintainability

REASON

There is no reason to assign a variable to itself. It is either a redundant statement that should be removed, or it is a mistake where some other value was intended in the assignment.

EXAMPLE (BAD)

```
1  declare
2      l_function_result pls_integer;
3      l_parallel_degree pls_integer;
4  begin
5      l_function_result := maintenance.get_config('parallel_degree');
6      l_parallel_degree := l_parallel_degree;
7  end;
8  /
```

EXAMPLE (GOOD)

```
1  declare
2      l_function_result pls_integer;
3      l_parallel_degree pls_integer;
4  begin
5      l_function_result := maintenance.get_config('parallel_degree');
6      l_parallel_degree := l_function_result;
7  end;
8  /
```

G-2150: Avoid comparisons with NULL value, consider using IS [NOT] NULL.

Blocker

Portability, Reliability

REASON

The `null` value can cause confusion both from the standpoint of code review and code execution. You must always use the `is null` or `is not null` syntax when you need to check if a value is or is not `null`.

EXAMPLE (BAD)

```
1  declare
2      l_value integer;
3  begin
4      if l_value = null then
5          null;
6      end if;
7  end;
8  /
```

EXAMPLE (GOOD)

```
1  declare
2      l_value integer;
3  begin
4      if l_value is null then
5          null;
6      end if;
7  end;
8  /
```


G-2160: Avoid initializing variables using functions in the declaration section.

Critical

Reliability

REASON

If your initialization fails, you will not be able to handle the error in your exceptions block.

EXAMPLE (BAD)

```
1  declare
2      co_department_id constant integer := 100;
3      l_department_name departments.department_name%type :=
4          department_api.name_by_id(in_id => co_department_id);
5  begin
6      sys.dbms_output.put_line(l_department_name);
7  end;
8  /
```

EXAMPLE (GOOD)

```
1  declare
2      co_department_id constant integer := 100;
3      co_unkown_name constant departments.department_name%type := 'unknown';
4      l_department_name departments.department_name%type;
5  begin
6      <<init>>
7      begin
8          l_department_name := department_api.name_by_id(in_id => co_department_id);
9      exception
10         when value_error then
11             l_department_name := co_unkown_name;
12     end init;
13
14     sys.dbms_output.put_line(l_department_name);
15 end;
16 /
```

G-2170: Never overload variables.

Major

Reliability

REASON

The readability of your code will be higher when you do not overload variables.

EXAMPLE (BAD)

```
1  begin
2      <<main>>
3      declare
4          co_main constant user_objects.object_name%type := 'test_main';
5          co_sub  constant user_objects.object_name%type := 'test_sub';
6          co_sep  constant user_objects.object_name%type := ' - ';
7          l_variable user_objects.object_name%type := co_main;
8      begin
9          <<sub>>
10         declare
11             l_variable user_objects.object_name%type := co_sub;
12         begin
13             sys.dbms_output.put_line(l_variable || co_sep || main.l_variable);
14         end sub;
15     end main;
16 end;
17 /
```

EXAMPLE (GOOD)

```
1  begin
2      <<main>>
3      declare
4          co_main constant user_objects.object_name%type := 'test_main';
5          co_sub  constant user_objects.object_name%type := 'test_sub';
6          co_sep  constant user_objects.object_name%type := ' - ';
7          l_main_variable user_objects.object_name%type := co_main;
8      begin
9          <<sub>>
10         declare
11             l_sub_variable user_objects.object_name%type := co_sub;
12         begin
13             sys.dbms_output.put_line(l_sub_variable || co_sep || l_main_variable);
14         end sub;
15     end main;
16 end;
17 /
```

G-2180: Never use quoted identifiers.

Major

Maintainability

REASON

Quoted identifiers make your code hard to read and maintain.

EXAMPLE (BAD)

```
1  declare
2      "sal+comm" integer;
3      "my constant" constant integer := 1;
4      "my exception" exception;
5  begin
6      "sal+comm" := "my constant";
7  exception
8      when "my exception" then
9          null;
10 end;
11 /
```

EXAMPLE (GOOD)

```
1  declare
2      l_sal_comm      integer;
3      co_my_constant constant integer := 1;
4      e_my_exception exception;
5  begin
6      l_sal_comm := co_my_constant;
7  exception
8      when e_my_exception then
9          null;
10 end;
11 /
```

G-2185: Avoid using overly short names for explicitly or implicitly declared identifiers.

Minor

Maintainability

REASON

You should ensure that the name you have chosen well defines its purpose and usage. While you can save a few keystrokes typing very short names, the resulting code is obscure and hard for anyone besides the author to understand.

EXAMPLE (BAD)

```
1  declare
2      i integer;
3      c constant integer := 1;
4      e exception;
5  begin
6      i := c;
7  exception
8      when e then
9          null;
10 end;
11 /
```

EXAMPLE (GOOD)

```
1  declare
2      l_sal_comm integer;
3      co_my_constant constant integer := 1;
4      e_my_exception exception;
5  begin
6      l_sal_comm := co_my_constant;
7  exception
8      when e_my_exception then
9          null;
10 end;
11 /
```

G-2190: Avoid using ROWID or UROWID.

Major

Portability, Reliability

REASON

Be careful about your use of Oracle-specific data types like `rowid` and `urowid`. They might offer a slight improvement in performance over other means of identifying a single row (primary key or unique index value), but that is by no means guaranteed.

Use of `rowid` or `urowid` means that your SQL statement will not be portable to other SQL databases. Many developers are also not familiar with these data types, which can make the code harder to maintain.

EXAMPLE (BAD)

```
1  declare
2      l_department_name departments.department_name%type;
3      l_rowid rowid;
4  begin
5      update departments
6          set department_name = l_department_name
7          where rowid = l_rowid;
8  end;
9  /
```

EXAMPLE (GOOD)

```
1  declare
2      l_department_name departments.department_name%type;
3      l_department_id departments.department_id%type;
4  begin
5      update departments
6          set department_name = l_department_name
7          where department_id = l_department_id;
8  end;
9  /
```

Numeric Data Types

G-2210: Avoid declaring NUMBER variables, constants or subtypes with no precision.

Minor

Efficiency

REASON

If you do not specify precision `number` is defaulted to 38 or the maximum supported by your system, whichever is less. You may well need all this precision, but if you know you do not, you should specify whatever matches your needs.

EXAMPLE (BAD)

```
1  create or replace package body constants_up is
2      co_small_increase constant number := 0.1;
3
4      function small_increase return number is
5      begin
6          return co_small_increase;
7      end small_increase;
8  end constants_up;
9  /
```

EXAMPLE (GOOD)

```
1  create or replace package body constants_up is
2      co_small_increase constant number(5,1) := 0.1;
3
4      function small_increase return number is
5      begin
6          return co_small_increase;
7      end small_increase;
8  end constants_up;
9  /
```

G-2220: Try to use PLS_INTEGER instead of NUMBER for arithmetic operations with integer values.

Minor

Efficiency

REASON

`pls_integer` having a length of -2,147,483,648 to 2,147,483,647, on a 32bit system.

There are many reasons to use `pls_integer` instead of `number` :

- `pls_integer` uses less memory
- `pls_integer` uses machine arithmetic, which is up to three times faster than library arithmetic, which is used by `number` .

EXAMPLE (BAD)

```
1  create or replace package body constants_up is
2      co_big_increase constant number(5,0) := 1;
3
4      function big_increase return number is
5      begin
6          return co_big_increase;
7      end big_increase;
8  end constants_up;
9  /
```

EXAMPLE (GOOD)

```
1  create or replace package body constants_up is
2      co_big_increase constant pls_integer := 1;
3
4      function big_increase return pls_integer is
5      begin
6          return co_big_increase;
7      end big_increase;
8  end constants_up;
9  /
```

G-2230: Try to use SIMPLE_INTEGER datatype when appropriate.

Minor

Efficiency

RESTRICTION

ORACLE 11g or later

REASON

`simple_integer` does no checks on numeric overflow, which results in better performance compared to the other numeric datatypes.

With ORACLE 11g, the new data type `simple_integer` has been introduced. It is a sub-type of `pls_integer` and covers the same range. The basic difference is that `simple_integer` is always `not null`. When the value of the declared variable is never going to be null then you can declare it as `simple_integer`. Another major difference is that you will never face a numeric overflow using `simple_integer` as this data type wraps around without giving any error.

`simple_integer` data type gives major performance boost over `pls_integer` when code is compiled in `native` mode, because arithmetic operations on `simple_integer` type are performed directly at the hardware level.

EXAMPLE (BAD)

```
1  create or replace package body constants_up is
2      co_big_increase constant number(5,0) := 1;
3
4      function big_increase return number is
5      begin
6          return co_big_increase;
7      end big_increase;
8  end constants_up;
9  /
```

EXAMPLE (GOOD)

```
1  create or replace package body constants_up is
2      co_big_increase constant simple_integer := 1;
3
4      function big_increase return simple_integer is
5      begin
6          return co_big_increase;
7      end big_increase;
8  end constants_up;
9  /
```


Character Data Types

G-2310: Avoid using CHAR data type.

Major

Reliability

REASON

`char` is a fixed length data type, which should only be used when appropriate. `char` columns/variables are always filled to its specified lengths; this may lead to unwanted side effects and undesired results.

EXAMPLE (BAD)

```
1  create or replace package types_up
2  is
3      subtype description_type is char(200);
4  end types_up;
5  /
```

EXAMPLE (GOOD)

```
1  create or replace package types_up
2  is
3      subtype description_type is varchar2(200 char);
4  end types_up;
5  /
```

G-2320: Never use VARCHAR data type.

Major

Portability

REASON

Do not use the `varchar` data type. Use the `varchar2` data type instead. Although the `varchar` data type is currently synonymous with `varchar2`, the `varchar` data type is scheduled to be redefined as a separate data type used for variable-length character strings compared with different comparison semantics.

EXAMPLE (BAD)

```
1  create or replace package types_up is
2      subtype description_type is varchar(200);
3  end types_up;
4  /
```

EXAMPLE (GOOD)

```
1  create or replace package types_up is
2      subtype description_type is varchar2(200 char);
3  end types_up;
4  /
```

G-2330: Never use zero-length strings to substitute NULL.

Major

Portability

REASON

Today zero-length strings and `null` are currently handled identical by ORACLE. There is no guarantee that this will still be the case in future releases, therefore if you mean `null` use `null`.

EXAMPLE (BAD)

```
1  create or replace package body constants_up is
2      co_null_string constant varchar2(1) := '';
3
4      function null_string return varchar2 is
5      begin
6          return co_null_string;
7      end null_string;
8  end constants_up;
9  /
```

EXAMPLE (GOOD)

```
1  create or replace package body constants_up is
2
3      function empty_string return varchar2 is
4      begin
5          return null;
6      end empty_string;
7  end constants_up;
8  /
```

G-2340: Always define your VARCHAR2 variables using CHAR SEMANTIC (if not defined anchored).

 Minor

Reliability

REASON

Changes to the `nls_length_semantic` will only be picked up by your code after a recompilation.

In a multibyte environment a `varchar2(10)` definition may not necessarily hold 10 characters when multibyte characters are part of the value that should be stored, unless the definition was done using the `char` semantic.

EXAMPLE (BAD)

```
1  create or replace package types_up is
2      subtype description_type is varchar2(200);
3  end types_up;
4  /
```

EXAMPLE (GOOD)

```
1  create or replace package types_up is
2      subtype description_type is varchar2(200 char);
3  end types_up;
4  /
```

Boolean Data Types

G-2410: Try to use boolean data type for values with dual meaning.

Minor

Maintainability

REASON

The use of `true` and `false` clarifies that this is a boolean value and makes the code easier to read.

EXAMPLE (BAD)

```
1  declare
2      co_newfile constant pls_integer := 1000;
3      co_oldfile constant pls_integer := 500;
4      l_bigger pls_integer;
5  begin
6      if co_newfile < co_oldfile then
7          l_bigger := constants_up.co_numeric_true;
8      else
9          l_bigger := constants_up.co_numeric_false;
10     end if;
11 end;
12 /
```

EXAMPLE (BETTER)

```
1  declare
2      co_newfile constant pls_integer := 1000;
3      co_oldfile constant pls_integer := 500;
4      l_bigger boolean;
5  begin
6      if co_newfile < co_oldfile then
7          l_bigger := true;
8      else
9          l_bigger := false;
10     end if;
11 end;
12 /
```

EXAMPLE (GOOD)

```
1  declare
2      co_newfile constant pls_integer := 1000;
3      co_oldfile constant pls_integer := 500;
4      l_bigger boolean;
5  begin
6      l_bigger := nvl(co_newfile < co_oldfile, false);
7  end;
8  /
```

Large Objects

G-2510: Avoid using the LONG and LONG RAW data types.

Major

Portability

REASON

`long` and `long raw` data types have been deprecated by ORACLE since version 8i - support might be discontinued in future ORACLE releases.

There are many constraints to `long` datatypes in comparison to the `lob` types.

EXAMPLE (BAD)

```
1  create or replace package example_package is
2      g_long long;
3      g_raw  long raw;
4
5      procedure do_something;
6  end example_package;
7  /
8
9  create or replace package body example_package is
10     procedure do_something is
11     begin
12         null;
13     end do_something;
14 end example_package;
15 /
```

EXAMPLE (GOOD)

```
1  create or replace package example_package is
2      procedure do_something;
3  end example_package;
4  /
5
6  create or replace package body example_package is
7      g_long clob;
8      g_raw  blob;
9
10     procedure do_something is
11     begin
12         null;
13     end do_something;
14 end example_package;
15 /
```

Cursor Variables

G-2610: Never use self-defined weak ref cursor types.

Minor

Changeability, Maintainability, Portability, Reusability

REASON

There is no reason to define your own weak ref cursor types, as they are not different from the built-in `sys_refcursor`. Introducing your own types just gives you unnecessary maintenance to perform.

EXAMPLE (BAD)

```
1  declare
2      type local_weak_cursor_type is ref cursor;
3      cv_data local_weak_cursor_type;
4  begin
5      if configuration.use_employee then
6          open cv_data for
7              select e.employee_id, e.first_name, e.last_name
8                  from employees e;
9      else
10         open cv_data for
11             select e.emp_id, e.name
12                 from emp e;
13     end if;
14 end;
15 /
```

EXAMPLE (GOOD)

```
1  declare
2      cv_data sys_refcursor;
3  begin
4      if configuration.use_employee then
5          open cv_data for
6              select e.employee_id, e.first_name, e.last_name
7                  from employees e;
8      else
9          open cv_data for
10             select e.emp_id, e.name
11                 from emp e;
12     end if;
13 end;
14 /
```

DML & SQL

General

G-3110: Always specify the target columns when coding an insert statement.

Major

Maintainability, Reliability

REASON

Data structures often change. Having the target columns in your insert statements will lead to change-resistant code.

EXAMPLE (BAD)

```
1  insert into departments
2      values (departments_seq.nextval
3              , 'Support'
4              , 100
5              , 10);
```

EXAMPLE (GOOD)

```
1  insert into departments (department_id
2                          , department_name
3                          , manager_id
4                          , location_id)
5      values (departments_seq.nextval
6              , 'Support'
7              , 100
8              , 10);
```


G-3115: Avoid self-assigning a column.

 **Minor**

Maintainability

REASON

There is normally no reason to assign a column to itself. It is either a redundant statement that should be removed, or it is a mistake where some other value was intended in the assignment.

One exception to this rule can be when you attempt to fire cross edition triggers when using Edition Based Redefinition.

EXAMPLE (BAD)

```
1  update employees
2    set first_name = first_name;
```

EXAMPLE (GOOD)

```
1  update employees
2    set first_name = initcap(first_name);
```

G-3120: Always use table aliases when your SQL statement involves more than one source.

Major

Maintainability

REASON

It is more human readable to use aliases instead of writing columns with no table information.

Especially when using subqueries the omission of table aliases may end in unexpected behavior and result.

EXAMPLE (BAD)

```
1  select last_name
2         ,first_name
3         ,department_name
4  from     employees
5         join departments using (department_id)
6  where extract(month from hire_date) = extract(month from sysdate);
```

EXAMPLE (BETTER)

```
1  select e.last_name
2         ,e.first_name
3         ,d.department_name
4  from     employees e
5         join departments d on (e.department_id = d.department_id)
6  where extract(month from e.hire_date) = extract(month from sysdate);
```

EXAMPLE (GOOD)

Using meaningful aliases improves the readability of your code.

```
1  select emp.last_name
2         ,emp.first_name
3         ,dept.department_name
4  from     employees emp
5         join departments dept on (emp.department_id = dept.department_id)
6  where extract(month from emp.hire_date) = extract(month from sysdate);
```

EXAMPLE SUBQUERY (BAD)

If the `jobs` table has no `employee_id` column and `employees` has one this query will not raise an error but return all rows of the `employees` table as a subquery is allowed to access columns of all its parent tables - this construct is known as correlated subquery.

```
1  select last_name
2         ,first_name
3  from employees
4  where employee_id in (select employee_id
5                          from jobs
6                          where job_title like '%Manager%');
```

EXAMPLE SUBQUERY (GOOD)

If the `jobs` table has no `employee_id` column this query will return an error due to the directive (given by adding the

table alias to the column) to read the `employee_id` column from the `jobs` table.

```
1  select emp.last_name
2         ,emp.first_name
3  from employees emp
4  where emp.employee_id in (select j.employee_id
5                             from jobs j
6                             where j.job_title like '%Manager%');
```

G-3130: Try to use ANSI SQL-92 join syntax.

Minor

Maintainability, Portability

REASON

ANSI SQL-92 join syntax supports the full outer join. A further advantage of the ANSI SQL-92 join syntax is the separation of the join condition from the query filters.

EXAMPLE (BAD)

```
1  select e.employee_id
2         ,e.last_name
3         ,e.first_name
4         ,d.department_name
5  from employees e
6         ,departments d
7  where e.department_id = d.department_id
8         and extract(month from e.hire_date) = extract(month from sysdate);
```

EXAMPLE (GOOD)

```
1  select emp.employee_id
2         ,emp.last_name
3         ,emp.first_name
4         ,dept.department_name
5  from      employees emp
6         join departments dept on dept.department_id = emp.department_id
7  where extract(month from emp.hire_date) = extract(month from sysdate);
```

G-3140: Try to use anchored records as targets for your cursors.

Major

Maintainability, Reliability

REASON

Using cursor-anchored records as targets for your cursors results enables the possibility of changing the structure of the cursor without regard to the target structure.

EXAMPLE (BAD)

```
1  declare
2      cursor c_employees is
3          select employee_id, first_name, last_name
4              from employees;
5      l_employee_id employees.employee_id%type;
6      l_first_name  employees.first_name%type;
7      l_last_name   employees.last_name%type;
8  begin
9      open c_employees;
10     fetch c_employees into l_employee_id, l_first_name, l_last_name;
11     <<process_employees>>
12     while c_employees%found
13     loop
14         -- do something with the data
15         fetch c_employees into l_employee_id, l_first_name, l_last_name;
16     end loop process_employees;
17     close c_employees;
18 end;
19 /
```

EXAMPLE (GOOD)

```
1  declare
2      cursor c_employees is
3          select employee_id, first_name, last_name
4              from employees;
5      r_employee c_employees%rowtype;
6  begin
7      open c_employees;
8      fetch c_employees into r_employee;
9      <<process_employees>>
10     while c_employees%found
11     loop
12         -- do something with the data
13         fetch c_employees into r_employee;
14     end loop process_employees;
15     close c_employees;
16 end;
17 /
```

G-3150: Try to use identity columns for surrogate keys.

Minor

Maintainability, Reliability

RESTRICTION

ORACLE 12c

REASON

An identity column is a surrogate key by design – there is no reason why we should not take advantage of this natural implementation when the keys are generated on database level. Using identity column (and therefore assigning sequences as default values on columns) has a huge performance advantage over a trigger solution.

EXAMPLE (BAD)

```
1  create table locations (
2      location_id          number(10)          not null
3      ,location_name       varchar2(60 char) not null
4      ,city                varchar2(30 char) not null
5      ,constraint locations_pk primary key (location_id)
6  )
7  /
8
9  create sequence location_seq start with 1 cache 20
10 /
11
12 create or replace trigger location_br_i
13     before insert on locations
14     for each row
15 begin
16     :new.location_id := location_seq.nextval;
17 end;
18 /
```

EXAMPLE (GOOD)

```
1  create table locations (
2      location_id          number(10) generated always as identity
3      ,location_name       varchar2(60 char) not null
4      ,city                varchar2(30 char) not null
5      ,constraint locations_pk primary key (location_id))
6  /
```

`generated always as identity` ensures that the `location_id` is populated by a sequence. It is not possible to override the behavior in the application.

However, if you use a framework that produces an `insert` statement including the surrogate key column, and you cannot change this behavior, then you have to use the `generated by default on null as identity` option. This has the downside that the application may pass a value, which might lead to an immediate or delayed `ORA-00001: unique constraint violated` error.

G-3160: Avoid visible virtual columns.

Major

Maintainability, Reliability

Unsupported in PL/SQL Cop Validators

We cannot identify the type of a column. Requires `create table` and `alter table` parser support or access to the Oracle Data Dictionary.

RESTRICTION

ORACLE 12c

REASON

In contrast to visible columns, invisible columns are not part of a record defined using `%rowtype` construct. This is helpful as a virtual column may not be programmatically populated. If your virtual column is visible you have to manually define the record types used in API packages to be able to exclude them from being part of the record definition.

Invisible columns may be accessed by explicitly adding them to the column list in a `select` statement.

EXAMPLE (BAD)

```
1  alter table employees
2      add total_salary generated always as (salary + nvl(commission_pct,0) * salary)
3  /
4
5  declare
6      r_employee employees%rowtype;
7      l_id employees.employee_id%type := 107;
8  begin
9      r_employee := employee_api.employee_by_id(l_id);
10     r_employee.salary := r_employee.salary * constants_up.small_increase();
11
12     update employees
13         set row = r_employee
14         where employee_id = l_id;
15 end;
16 /
17
18 Error report -
19 ORA-54017: update operation disallowed on virtual columns
20 ORA-06512: at line 9
```

EXAMPLE (GOOD)

```
1  alter table employees
2      add total_salary invisible generated always as
3      (salary + nvl(commission_pct,0) * salary)
4  /
5
6  declare
7      r_employee employees%rowtype;
8      co_id constant employees.employee_id%type := 107;
9  begin
10     r_employee := employee_api.employee_by_id(co_id);
11     r_employee.salary := r_employee.salary * constants_up.small_increase();
12
13     update employees
14         set row = r_employee
15         where employee_id = co_id;
16 end;
17 /
```


G-3170: Always use DEFAULT ON NULL declarations to assign default values to table columns if you refuse to store NULL values.

Major

Reliability

Unsupported in PL/SQL Cop Validators

We cannot identify column default values. Requires `create table` and `alter table` parser support or access to the Oracle Data Dictionary.

RESTRICTION

ORACLE 12c

REASON

Default values have been nullifiable until ORACLE 12c. Meaning any tool sending null as a value for a column having a default value bypassed the default value. Starting with ORACLE 12c default definitions may have an `on null` definition in addition, which will assign the default value in case of a `null` value too.

EXAMPLE (BAD)

```
1  create table null_test (  
2      test_case          number(2) not null  
3      ,column_defaulted varchar2(10 char) default 'Default')  
4  /  
5  insert into null_test(test_case, column_defaulted) values (1,'Value');  
6  insert into null_test(test_case, column_defaulted) values (2,default);  
7  insert into null_test(test_case, column_defaulted) values (3,null);  
8  
9  select * from null_test;  
10  
11 TEST_CASE  COLUMN_DEF  
12 -----  -  
13          1 Value  
14          2 Default  
15          3
```

EXAMPLE (GOOD)

```

1  create table null_test (
2      test_case          number(2) not null
3      ,column_defaulted varchar2(10 char) default on null 'Default')
4  /
5  insert into null_test(test_case, column_defaulted) values (1, 'Value');
6  insert into null_test(test_case, column_defaulted) values (2, default);
7  insert into null_test(test_case, column_defaulted) values (3, null);
8
9  select * from null_test;
10
11  TEST_CASE COLUMN_DEF
12  -----
13           1 Value
14           2 Default
15           3 Default

```

G-3180: Always specify column names instead of positional references in ORDER BY clauses.

Major

Changeability, Reliability

REASON

If you change your `select` list afterwards the `order by` will still work but order your rows differently, when not changing the positional number. Furthermore, it is not comfortable to the readers of the code, if they have to count the columns in the `select` list to know the way the result is ordered.

EXAMPLE (BAD)

```
1  select upper(first_name)
2      ,last_name
3      ,salary
4      ,hire_date
5  from employees
6  order by 4,1,3;
```

EXAMPLE (GOOD)

```
1  select upper(first_name) as first_name
2      ,last_name
3      ,salary
4      ,hire_date
5  from employees
6  order by hire_date
7      ,first_name
8      ,salary;
```

G-3185: Never use ROWNUM at the same query level as ORDER BY.

Major

Reliability, Testability

REASON

The `rownum` pseudo-column is assigned before the `order by` clause is used, so using `rownum` on the same query level as `order by` will not assign numbers in the desired ordering. Instead you should move the `order by` into an inline view and use `rownum` in the outer query.

EXAMPLE (BAD)

```
1  select first_name
2         ,last_name
3         ,salary
4         ,hire_date
5         ,rownum as salary_rank
6  from employees
7  where rownum <= 5
8  order by salary desc;
```

EXAMPLE (GOOD)

```
1  select first_name
2         ,last_name
3         ,salary
4         ,hire_date
5         ,rownum as salary_rank
6  from (
7    select first_name
8           ,last_name
9           ,salary
10          ,hire_date
11    from employees
12    order by salary desc
13  )
14  where rownum <= 5;
```

G-3190: Avoid using NATURAL JOIN.

Major

Changeability, Reliability

REASON

A `natural join` joins tables on equally named columns. This may comfortably fit on first sight, but adding logging columns to a table (`changed_by` , `changed_date`) will result in inappropriate join conditions.

EXAMPLE (BAD)

```
1  select department_name
2      ,last_name
3      ,first_name
4  from employees natural join departments
5  order by department_name
6          ,last_name;
7
8  DEPARTMENT_NAME          LAST_NAME          FIRST_NAME
9  -----
10 Accounting              Gietz              William
11 Executive                De Haan            Lex
12 ...
13
14 alter table departments add modified_at date default on null sysdate;
15 alter table employees add modified_at date default on null sysdate;
16
17 select department_name
18      ,last_name
19      ,first_name
20  from employees natural join departments
21  order by department_name
22          ,last_name;
23
24 No data found
```

EXAMPLE (GOOD)

```
1  select d.department_name
2      ,e.last_name
3      ,e.first_name
4  from employees e
5  join departments d on (e.department_id = d.department_id)
6  order by d.department_name
7          ,e.last_name;
8
9  DEPARTMENT_NAME          LAST_NAME          FIRST_NAME
10 -----
11 Accounting              Gietz              William
12 Executive                De Haan            Lex
13 ...
```

G-3195: Always use wildcards in a LIKE clause.

Minor

Maintainability

REASON

Using `like` without at least one wildcard (`%` or `_`) is unclear to a maintainer whether a wildcard is forgotten or it is meant as equality test. A common antipattern is also to forget that an underscore is a wildcard, so using `like` instead of `=` can return unwanted rows. If the `char` datatype is involved, there is also the danger of `like` not using blank padded comparison where `=` will. Depending on use case, you should either remember at least one wildcard or use normal equality operator.

EXAMPLE (BAD)

```
1 select e.employee_id
2       ,e.last_name
3   from employees e
4  where e.last_name like 'Smith';
```

EXAMPLE (GOOD)

```
1 select e.employee_id
2       ,e.last_name
3   from employees e
4  where e.last_name like 'Smith%';
```

EXAMPLE (GOOD)

```
1 select e.employee_id
2       ,e.last_name
3   from employees e
4  where e.last_name = 'Smith';
```

Bulk Operations

G-3210: Always use BULK OPERATIONS (BULK COLLECT, FORALL) whenever you have to execute a DML statement for more than 4 times.

Major

Efficiency

REASON

Context switches between PL/SQL and SQL are extremely costly. BULK Operations reduce the number of switches by passing an array to the SQL engine, which is used to execute the given statements repeatedly.

(Depending on the PLSQL_OPTIMIZE_LEVEL parameter a conversion to BULK COLLECT will be done by the PL/SQL compiler automatically.)

EXAMPLE (BAD)

```
1  declare
2      t_employee_ids employee_api.t_employee_ids_type;
3      co_increase constant employees.salary%type := 0.1;
4      co_department_id constant departments.department_id%type := 10;
5  begin
6      t_employee_ids := employee_api.employee_ids_by_department(
7          id_in => co_department_id
8      );
9      <<process_employees>>
10     for i in 1..t_employee_ids.count()
11     loop
12         update employees
13             set salary = salary + (salary * co_increase)
14             where employee_id = t_employee_ids(i);
15     end loop process_employees;
16 end;
17 /
```

EXAMPLE (GOOD)

```
1  declare
2      t_employee_ids employee_api.t_employee_ids_type;
3      co_increase constant employees.salary%type := 0.1;
4      co_department_id constant departments.department_id%type := 10;
5  begin
6      t_employee_ids := employee_api.employee_ids_by_department(
7          id_in => co_department_id
8      );
9      <<process_employees>>
10     forall i in 1..t_employee_ids.count()
11         update employees
12             set salary = salary + (salary * co_increase)
13             where employee_id = t_employee_ids(i);
14 end;
15 /
```

Transaction Control

G-3310: Never commit within a cursor loop.

Critical

Efficiency, Reliability

REASON

Doing frequent commits within a cursor loop (all types of loops over cursors, whether implicit cursor for loop or loop with explicit fetch from cursor or cursor variable) risks not being able to complete due to ORA-01555, gives bad performance, and risks that the work is left in an unknown half-finished state and cannot be restarted.

- If the work belongs together (an atomic transaction) the `commit` should be moved to after the loop. Or even better if the logic can be rewritten to a single DML statement on all relevant rows instead of a loop, committing after the single statement.
- If each loop iteration is a self-contained atomic transaction, consider instead to populate a collection of transactions to be done (taking restartability into account by collection population), loop over that collection (instead of looping over a cursor) and call a procedure (that contains the transaction logic and the `commit`) in the loop (see also [G-3320](#)).

EXAMPLE (BAD)

```
1  declare
2      l_counter    integer := 0;
3      l_discount   discount.percentage%type;
4  begin
5      for r_order in (
6          select o.order_id, o.customer_id
7              from orders o
8              where o.order_status = 'New'
9      ) loop
10         l_discount := sales_api.calculate_discount(p_customer_id => r_order.customer_id);
11
12         update order_lines ol
13             set ol.discount = l_discount
14             where ol.order_id = r_order.order_id;
15
16         l_counter := l_counter + 1;
17         if l_counter = 100 then
18             commit;
19             l_counter := 0;
20         end if;
21     end loop;
22     if l_counter > 0 then
23         commit;
24     end if;
25 end;
26 /
```

EXAMPLE (GOOD)


```

1  declare
2      l_discount discount.percentage%type;
3  begin
4      for r_order in (
5          select o.order_id, o.customer_id
6              from orders o
7              where o.order_status = 'New'
8      ) loop
9          l_discount := sales_api.calculate_discount(p_customer_id => r_order.customer_id);
10
11         update order_lines ol
12             set ol.discount = l_discount
13             where ol.order_id = r_order.order_id;
14     end loop;
15
16     commit;
17 end;
18 /

```

EXAMPLE (BETTER)

(Assuming suitable foreign key relationship exists to allow updating a join.)

```

1  begin
2      update (
3          select o.customer_id, ol.discount
4              from orders o
5              join order_lines ol
6                  on ol.order_id = o.order_id
7              where o.order_status = 'New'
8      )
9      set discount = sales_api.calculate_discount(p_customer_id => customer_id);
10
11     commit;
12 end;
13 /

```

G-3320: Try to move transactions within a non-cursor loop into procedures.

Major

Maintainability, Reusability, Testability

REASON

Commit inside a non-cursor loop (other loop types than loops over cursors - see also [G-3310](#)) is either a self-contained atomic transaction, or it is a chunk (with suitable restartability handling) of very large data manipulations. In either case encapsulating the transaction in a procedure is good modularity, enabling reuse and testing of a single call.

EXAMPLE (BAD)

```
1  begin
2      for l_counter in 1..5 loop
3          insert into headers (id, text) values (l_counter, 'Number '||l_counter);
4
5          insert into lines (header_id, line_no, text)
6              select l_counter, rownum, 'Line '||rownum
7                  from dual
8                  connect by level <= 3;
9
10         commit;
11     end loop;
12 end;
13 /
```

EXAMPLE (GOOD)

```
1  declare
2      procedure create_rows (
3          p_header_id in headers.id%type
4      ) is
5      begin
6          insert into headers (id, text) values (p_header_id, 'Number '||p_header_id);
7
8          insert into lines (header_id, line_no, text)
9              select p_header_id, rownum, 'Line '||rownum
10                 from dual
11                 connect by level <= 3;
12
13         commit;
14     end;
15 begin
16     for l_counter in 1..5 loop
17         create_rows(l_counter);
18     end loop;
19 end;
20 /
```

Control Structures

CURSOR

G-4110: Always use %NOTFOUND instead of NOT %FOUND to check whether a cursor returned data.

 Minor

Maintainability

REASON

The readability of your code will be higher when you avoid negative sentences.

EXAMPLE (BAD)

```
1  declare
2      cursor c_employees is
3          select last_name
4                  ,first_name
5          from employees
6          where commission_pct is not null;
7
8      r_employee c_employees%rowtype;
9  begin
10     open c_employees;
11
12     <<read_employees>>
13     loop
14         fetch c_employees into r_employee;
15         exit read_employees when not c_employees%found;
16     end loop read_employees;
17
18     close c_employees;
19 end;
20 /
```

EXAMPLE (GOOD)

```
1  declare
2      cursor c_employees is
3          select last_name
4                  ,first_name
5          from employees
6          where commission_pct is not null;
7
8      r_employee c_employees%rowtype;
9  begin
10     open c_employees;
11
12     <<read_employees>>
13     loop
14         fetch c_employees into r_employee;
15         exit read_employees when c_employees%notfound;
16     end loop read_employees;
17
18     close c_employees;
19 end;
20 /
```

Critical

Reliability

REASON

`%notfound` is set to `true` as soon as less than the number of rows defined by the `limit` clause has been read.

EXAMPLE (BAD)

The employees table holds 107 rows. The example below will only show 100 rows as the cursor attribute `notfound` is set to true as soon as the number of rows to be fetched defined by the `limit` clause is not fulfilled anymore.

```
1  declare
2      cursor c_employees is
3          select *
4              from employees
5              order by employee_id;
6
7      type t_employees_type is table of c_employees%rowtype;
8      t_employees t_employees_type;
9      co_bulk_size constant simple_integer := 10;
10 begin
11     open c_employees;
12
13     <<process_employees>>
14     loop
15         fetch c_employees bulk collect into t_employees limit co_bulk_size;
16         exit process_employees when c_employees%notfound;
17
18         <<display_employees>>
19         for i in 1..t_employees.count()
20             loop
21                 sys.dbms_output.put_line(t_employees(i).last_name);
22             end loop display_employees;
23     end loop process_employees;
24
25     close c_employees;
26 end;
27 /
```

EXAMPLE (BETTER)

This example will show all 107 rows but execute one fetch too much (12 instead of 11).

```

1  declare
2      cursor c_employees is
3          select *
4              from employees
5              order by employee_id;
6
7      type t_employees_type is table of c_employees%rowtype;
8      t_employees t_employees_type;
9      co_bulk_size constant simple_integer := 10;
10 begin
11     open c_employees;
12
13     <<process_employees>>
14     loop
15         fetch c_employees bulk collect into t_employees limit co_bulk_size;
16         exit process_employees when t_employees.count() = 0;
17         <<display_employees>>
18         for i in 1..t_employees.count()
19             loop
20                 sys.dbms_output.put_line(t_employees(i).last_name);
21             end loop display_employees;
22     end loop process_employees;
23
24     close c_employees;
25 end;
26 /

```

EXAMPLE (GOOD)

This example does the trick (11 fetches only to process all rows)

```

1  declare
2      cursor c_employees is
3          select *
4              from employees
5              order by employee_id;
6
7      type t_employees_type is table of c_employees%rowtype;
8      t_employees t_employees_type;
9      co_bulk_size constant simple_integer := 10;
10 begin
11     open c_employees;
12
13     <<process_employees>>
14     loop
15         fetch c_employees bulk collect into t_employees limit co_bulk_size;
16         <<display_employees>>
17         for i in 1..t_employees.count()
18             loop
19                 sys.dbms_output.put_line(t_employees(i).last_name);
20             end loop display_employees;
21         exit process_employees when t_employees.count() <> co_bulk_size;
22     end loop process_employees;
23
24     close c_employees;
25 end;
26 /

```

G-4130: Always close locally opened cursors.

Major

Efficiency, Reliability

REASON

Any cursors left open can consume additional memory space (i.e. SGA) within the database instance, potentially in both the shared and private SQL pools. Furthermore, failure to explicitly close cursors may also cause the owning session to exceed its maximum limit of open cursors (as specified by the `open_cursors` database initialization parameter), potentially resulting in the Oracle error of “ORA-01000: maximum open cursors exceeded”.

EXAMPLE (BAD)

```
1  create or replace package body employee_api as
2      function department_salary (in_dept_id in departments.department_id%type)
3          return number is
4          cursor c_department_salary(p_dept_id in departments.department_id%type) is
5              select sum(salary) as sum_salary
6                  from employees
7                  where department_id = p_dept_id;
8          r_department_salary c_department_salary%rowtype;
9  begin
10     open c_department_salary(p_dept_id => in_dept_id);
11     fetch c_department_salary into r_department_salary;
12
13     return r_department_salary.sum_salary;
14 end department_salary;
15 end employee_api;
16 /
```

EXAMPLE (GOOD)

```
1  create or replace package body employee_api as
2      function department_salary (in_dept_id in departments.department_id%type)
3          return number is
4          cursor c_department_salary(p_dept_id in departments.department_id%type) is
5              select sum(salary) as sum_salary
6                  from employees
7                  where department_id = p_dept_id;
8          r_department_salary c_department_salary%rowtype;
9  begin
10     open c_department_salary(p_dept_id => in_dept_id);
11     fetch c_department_salary into r_department_salary;
12     close c_department_salary;
13     return r_department_salary.sum_salary;
14 end department_salary;
15 end employee_api;
16 /
```

G-4140: Avoid executing any statements between a SQL operation and the usage of an implicit cursor attribute.

Major

Reliability

REASON

Oracle provides a variety of cursor attributes (like `%found` and `%rowcount`) that can be used to obtain information about the status of a cursor, either implicit or explicit.

You should avoid inserting any statements between the cursor operation and the use of an attribute against that cursor. Interposing such a statement can affect the value returned by the attribute, thereby potentially corrupting the logic of your program.

In the following example, a procedure call is inserted between the `delete` statement and a check for the value of `sql%rowcount`, which returns the number of rows modified by that last SQL statement executed in the session. If this procedure includes a `commit` / `rollback` or another implicit cursor the value of `sql%rowcount` is affected.

EXAMPLE (BAD)

```
1  create or replace package body employee_api as
2      co_one constant simple_integer := 1;
3
4      procedure process_dept(in_dept_id in departments.department_id%type) is
5      begin
6          null;
7      end process_dept;
8
9      procedure remove_employee (in_employee_id in employees.employee_id%type) is
10         l_dept_id      employees.department_id%type;
11      begin
12          delete from employees
13              where employee_id = in_employee_id
14              returning department_id into l_dept_id;
15
16          process_dept(in_dept_id => l_dept_id);
17
18          if sql%rowcount > co_one then
19              -- too many rows deleted.
20              rollback;
21          end if;
22      end remove_employee;
23  end employee_api;
24  /
```

EXAMPLE (GOOD)

```

1  create or replace package body employee_api as
2      co_one constant simple_integer := 1;
3
4      procedure process_dept(in_dept_id in departments.department_id%type) is
5      begin
6          null;
7      end process_dept;
8
9      procedure remove_employee (in_employee_id in employees.employee_id%type) is
10         l_dept_id      employees.department_id%type;
11         l_deleted_emps simple_integer;
12     begin
13         delete from employees
14             where employee_id = in_employee_id
15             returning department_id into l_dept_id;
16
17         l_deleted_emps := sql%rowcount;
18
19         process_dept(in_dept_id => l_dept_id);
20
21         if l_deleted_emps > co_one then
22             -- too many rows deleted.
23             rollback;
24         end if;
25     end remove_employee;
26 end employee_api;
27 /

```


CASE / IF / DECODE / NVL / NVL2 / COALESCE

G-4210: Try to use CASE rather than an IF statement with multiple ELSIF paths.

Major

Maintainability, Testability

REASON

`if` statements containing multiple `elsif` tend to become complex quickly.

EXAMPLE (BAD)

```
1  declare
2      l_color varchar2(7 char);
3  begin
4      if l_color = constants_up.co_red then
5          my_package.do_red();
6      elsif l_color = constants_up.co_blue then
7          my_package.do_blue();
8      elsif l_color = constants_up.co_black then
9          my_package.do_black();
10     end if;
11 end;
12 /
```

EXAMPLE (GOOD)

```
1  declare
2      l_color types_up.color_code_type;
3  begin
4      case l_color
5          when constants_up.co_red then
6              my_package.do_red();
7          when constants_up.co_blue then
8              my_package.do_blue();
9          when constants_up.co_black then
10             my_package.do_black();
11         else null;
12     end case;
13 end;
14 /
```

G-4220: Try to use CASE rather than DECODE.

Minor

Maintainability, Portability

REASON

`decode` is an ORACLE specific function hard to understand and restricted to SQL only. The “newer” `case` function is much more common, has a better readability and may be used within PL/SQL too. Be careful that `decode` can handle `null` values, which the simple `case` cannot - for such cases you must use the searched `case` and `is null` instead.

EXAMPLE (BAD)

```
1  select decode(dummy, 'X', 1
2                      , 'Y', 2
3                      , 'Z', 3
4                      , 0)
5  from dual;
```

EXAMPLE (GOOD)

```
1  select case dummy
2         when 'X' then 1
3         when 'Y' then 2
4         when 'Z' then 3
5         else 0
6       end
7  from dual;
```

EXAMPLE (BAD)

```
1  select decode(dummy, 'X', 1
2                      , 'Y', 2
3                      , null, -1
4                      , 0)
5  from dual;
```

EXAMPLE (GOOD)

```
1  select case
2         when dummy = 'X' then 1
3         when dummy = 'Y' then 2
4         when dummy is null then -1
5         else 0
6       end
7  from dual;
```

G-4230: Always use a COALESCE instead of a NVL command, if parameter 2 of the NVL function is a function call or a SELECT statement.

 Critical

Efficiency, Reliability

REASON

The `nvl` function always evaluates both parameters before deciding which one to use. This can be harmful if parameter 2 is either a function call or a select statement, as it will be executed regardless of whether parameter 1 contains a `null` value or not.

The `coalesce` function does not have this drawback.

EXAMPLE (BAD)

```
1  select nvl(dummy, my_package.expensive_null(value_in => dummy))
2  from dual;
```

EXAMPLE (GOOD)

```
1  select coalesce(dummy, my_package.expensive_null(value_in => dummy))
2  from dual;
```

G-4240: Always use a CASE instead of a NVL2 command if parameter 2 or 3 of NVL2 is either a function call or a SELECT statement.

 Critical

Efficiency, Reliability

REASON

The `nvl2` function always evaluates all parameters before deciding which one to use. This can be harmful, if parameter 2 or 3 is either a function call or a select statement, as they will be executed regardless of whether parameter 1 contains a `null` value or not.

EXAMPLE (BAD)

```
1  select nvl2(dummy, my_package.expensive_nn(value_in => dummy),
2             my_package.expensive_null(value_in => dummy))
3  from dual;
```

EXAMPLE (GOOD)

```
1  select case
2         when dummy is null then
3             my_package.expensive_null(value_in => dummy)
4         else
5             my_package.expensive_nn(value_in => dummy)
6         end
7  from dual;
```

G-4250: Avoid using identical conditions in different branches of the same IF or CASE statement.

Major

Maintainability, Reliability, Testability

REASON

Conditions are evaluated top to bottom in branches of a `case` statement or chain of `if / elsif` statements. The first condition to evaluate as true leads to that branch being executed, the rest will never execute. Having an identical duplicated condition in another branch will never be reached and will be dead code.

EXAMPLE (BAD)

```
1  declare
2      l_color types_up.color_code_type;
3  begin
4      case l_color
5          when constants_up.co_red    then
6              my_package.do_red();
7          when constants_up.co_blue   then
8              my_package.do_blue();
9          when constants_up.co_red    then -- never reached
10             my_package.do_black();      -- dead code
11      else null;
12  end case;
13 end;
14 /
```

EXAMPLE (GOOD)

```
1  declare
2      l_color types_up.color_code_type;
3  begin
4      case l_color
5          when constants_up.co_red    then
6              my_package.do_red();
7          when constants_up.co_blue   then
8              my_package.do_blue();
9          when constants_up.co_black  then
10             my_package.do_black();
11      else null;
12  end case;
13 end;
14 /
```

G-4260: Avoid inverting boolean conditions with NOT.

Minor

Maintainability, Testability

REASON

It is more readable to use the opposite comparison operator instead of inverting the comparison with `not`.

EXAMPLE (BAD)

```
1  declare
2      l_color varchar2(7 char);
3  begin
4      if not l_color != constants_up.co_red then
5          my_package.do_red();
6      end if;
7  end;
8  /
```

EXAMPLE (GOOD)

```
1  declare
2      l_color types_up.color_code_type;
3  begin
4      if l_color = constants_up.co_red then
5          my_package.do_red();
6      end if;
7  end;
8  /
```

G-4270: Avoid comparing boolean values to boolean literals.

Minor

Maintainability, Testability

REASON

It is more readable to simply use the boolean value as a condition itself, rather than use a comparison condition comparing the boolean value to the literals `true` or `false`.

EXAMPLE (BAD)

```
1  declare
2      l_string    varchar2(10 char) := '42';
3      l_is_valid  boolean;
4  begin
5      l_is_valid := my_package.is_valid_number(l_string);
6      if l_is_valid = true then
7          my_package.convert_number(l_string);
8      end if;
9  end;
10 /
```

EXAMPLE (GOOD)

```
1  declare
2      l_string    varchar2(10 char) := '42';
3      l_is_valid  boolean;
4  begin
5      l_is_valid := my_package.is_valid_number(l_string);
6      if l_is_valid then
7          my_package.convert_number(l_string);
8      end if;
9  end;
10 /
```

Flow Control

G-4310: Never use GOTO statements in your code.

Major

Maintainability, Testability

REASON

Code containing gotos is hard to format. Indentation should be used to show logical structure, and gotos have an effect on logical structure. Using indentation to show the logical structure of a goto and its target, however, is difficult or impossible. (...)

Use of gotos is a matter of religion. My dogma is that in modern languages, you can easily replace nine out of ten gotos with equivalent sequential constructs. In these simple cases, you should replace gotos out of habit. In the hard cases, you can still exorcise the goto in nine out of ten cases: You can break the code into smaller routines, use try-finally, use nested ifs, test and retest a status variable, or restructure a conditional. Eliminating the goto is harder in these cases, but it's good mental exercise (...).

– McConnell, Steve C. (2004). *Code Complete. Second Edition*. Microsoft Press.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure password_check (in_password in varchar2) is
3          co_digitarray constant string(10 char) := '0123456789';
4          co_lower_bound constant simple_integer := 1;
5          co_errno        constant simple_integer := -20501;
6          co_errmsg       constant string(100 char) := 'Password must contain a digit.';
7          l_isdigit       boolean := false;
8          l_len_pw        pls_integer;
9          l_len_array     pls_integer;
10     begin
11         l_len_pw := length(in_password);
12         l_len_array := length(co_digitarray);
13
14         <<check_digit>>
15         for i in co_lower_bound .. l_len_array
16             loop
17                 <<check_pw_char>>
18                 for j in co_lower_bound .. l_len_pw
19                     loop
20                         if substr(in_password, j, 1) = substr(co_digitarray, i, 1) then
21                             l_isdigit := true;
22                             goto check_other_things;
23                         end if;
24                     end loop check_pw_char;
25             end loop check_digit;
26
27         <<check_other_things>>
28         null;
29
30         if not l_isdigit then
31             raise_application_error(co_errno, co_errmsg);
32         end if;
33     end password_check;
34 end my_package;
35 /
```


EXAMPLE (BETTER)

```
1  create or replace package body my_package is
2      procedure password_check (in_password in varchar2) is
3          co_digitarray constant string(10 char) := '0123456789';
4          co_lower_bound constant simple_integer := 1;
5          co_errno constant simple_integer := -20501;
6          co_errmsg constant string(100 char) := 'Password must contain a digit.';
7          l_isdigit boolean := false;
8          l_len_pw pls_integer;
9          l_len_array pls_integer;
10     begin
11         l_len_pw := length(in_password);
12         l_len_array := length(co_digitarray);
13
14         <<check_digit>>
15         for i in co_lower_bound .. l_len_array
16             loop
17                 <<check_pw_char>>
18                 for j in co_lower_bound .. l_len_pw
19                     loop
20                         if substr(in_password, j, 1) = substr(co_digitarray, i, 1) then
21                             l_isdigit := true;
22                             exit check_digit; -- early exit condition
23                         end if;
24                     end loop check_pw_char;
25                 end loop check_digit;
26
27                 <<check_other_things>>
28                 null;
29
30                 if not l_isdigit then
31                     raise_application_error(co_errno, co_errmsg);
32                 end if;
33             end password_check;
34     end my_package;
35 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      procedure password_check (in_password in varchar2) is
3          co_digitpattern constant string(10 char) := '\d';
4          co_errno constant simple_integer := -20501;
5          co_errmsg constant string(100 char) := 'Password must contain a digit.';
6      begin
7          if not regexp_like(in_password, co_digitpattern)
8              then
9              raise_application_error(co_errno, co_errmsg);
10         end if;
11     end password_check;
12 end my_package;
13 /
```

G-4320: Always label your loops.

Minor

Maintainability

REASON

It's a good alternative for comments to indicate the start and end of a named loop processing.

EXAMPLE (BAD)

```
1  declare
2      i integer;
3      co_min_value constant simple_integer := 1;
4      co_max_value constant simple_integer := 10;
5      co_increment constant simple_integer := 1;
6  begin
7      i := co_min_value;
8      while (i <= co_max_value)
9      loop
10         i := i + co_increment;
11     end loop;
12
13     loop
14         exit;
15     end loop;
16
17     for i in co_min_value..co_max_value
18     loop
19         sys.dbms_output.put_line(i);
20     end loop;
21
22     for r_employee in (select last_name from employees)
23     loop
24         sys.dbms_output.put_line(r_employee.last_name);
25     end loop;
26 end;
27 /
```

EXAMPLE (GOOD)

```

1  declare
2      i integer;
3      co_min_value constant simple_integer := 1;
4      co_max_value constant simple_integer := 10;
5      co_increment constant simple_integer := 1;
6  begin
7      i := co_min_value;
8      <<while_loop>>
9      while (i <= co_max_value)
10     loop
11         i := i + co_increment;
12     end loop while_loop;
13
14     <<basic_loop>>
15     loop
16         exit basic_loop;
17     end loop basic_loop;
18
19     <<for_loop>>
20     for i in co_min_value..co_max_value
21     loop
22         sys.dbms_output.put_line(i);
23     end loop for_loop;
24
25     <<process_employees>>
26     for r_employee in (select last_name
27                        from employees)
28     loop
29         sys.dbms_output.put_line(r_employee.last_name);
30     end loop process_employees;
31 end;
32 /

```

G-4325: Never reuse labels in inner scopes.

Major

Maintainability, Reliability, Testability

REASON

Reusing labels inside the scope of another label with the same name leads to confusion, less chance of understanding the code, and could lead to bugs (for example if using `exit my_label` exits at a different nesting level than expected.)

EXAMPLE (BAD)

```
1  <<my_label>>
2  declare
3      co_min_value constant simple_integer := 1;
4      co_max_value constant simple_integer := 8;
5  begin
6      <<my_label>>
7      for i in co_min_value..co_max_value
8      loop
9          sys.dbms_output.put_line(i);
10     end loop;
11 end;
12 /
```

EXAMPLE (GOOD)

```
1  <<output_values>>
2  declare
3      co_min_value constant simple_integer := 1;
4      co_max_value constant simple_integer := 8;
5  begin
6      <<process_values>>
7      for i in co_min_value..co_max_value
8      loop
9          sys.dbms_output.put_line(i);
10     end loop process_values;
11 end output_values;
12 /
```

G-4330: Always use a CURSOR FOR loop to process the complete cursor results unless you are using bulk operations.

Minor

Maintainability

REASON

It is easier for the reader to see, that the complete data set is processed. Using SQL to define the data to be processed is easier to maintain and typically faster than using conditional processing within the loop.

Since an `exit` statement is similar to a `goto` statement, it should be avoided, whenever possible.

EXAMPLE (BAD)

```
1  declare
2      cursor c_employees is
3          select employee_id, last_name
4              from employees;
5      r_employee c_employees%rowtype;
6  begin
7      open c_employees;
8
9      <<read_employees>>
10     loop
11         fetch c_employees into r_employee;
12         exit read_employees when c_employees%notfound;
13         sys.dbms_output.put_line(r_employee.last_name);
14     end loop read_employees;
15
16     close c_employees;
17 end;
18 /
```

EXAMPLE (GOOD)

```
1  declare
2      cursor c_employees is
3          select employee_id, last_name
4              from employees;
5  begin
6      <<read_employees>>
7      for r_employee in c_employees
8      loop
9          sys.dbms_output.put_line(r_employee.last_name);
10     end loop read_employees;
11 end;
12 /
```

G-4340: Always use a NUMERIC FOR loop to process a dense array.

Minor

Maintainability

REASON

It is easier for the reader to see, that the complete array is processed.

Since an `exit` statement is similar to a `goto` statement, it should be avoided, whenever possible.

EXAMPLE (BAD)

```
1  declare
2      type t_employee_type is varray(10) of employees.employee_id%type;
3      t_employees t_employee_type;
4      co_himuro      constant integer := 118;
5      co_livingston  constant integer := 177;
6      co_min_value   constant simple_integer := 1;
7      co_increment   constant simple_integer := 1;
8      i pls_integer;
9  begin
10     t_employees := t_employee_type(co_himuro, co_livingston);
11     i           := co_min_value;
12
13     <<process_employees>>
14     loop
15         exit process_employees when i > t_employees.count();
16         sys.dbms_output.put_line(t_employees(i));
17         i := i + co_increment;
18     end loop process_employees;
19 end;
20 /
```

EXAMPLE (GOOD)

```
1  declare
2      type t_employee_type is varray(10) of employees.employee_id%type;
3      t_employees t_employee_type;
4      co_himuro      constant integer := 118;
5      co_livingston  constant integer := 177;
6  begin
7      t_employees := t_employee_type(co_himuro, co_livingston);
8
9      <<process_employees>>
10     for i in 1..t_employees.count()
11     loop
12         sys.dbms_output.put_line(t_employees(i));
13     end loop process_employees;
14 end;
15 /
```

Major

Reliability

REASON

Doing so will not raise a `value_error` if the array you are looping through is empty. If you want to use `first()..last()` you need to check the array for emptiness beforehand to avoid the raise of `value_error`.

EXAMPLE (BAD)

```
1  declare
2      type t_employee_type is table of employees.employee_id%type;
3      t_employees t_employee_type := t_employee_type();
4  begin
5      <<process_employees>>
6      for i in t_employees.first()..t_employees.last()
7      loop
8          sys.dbms_output.put_line(t_employees(i)); -- some processing
9      end loop process_employees;
10 end;
11 /
```

EXAMPLE (BETTER)

Raise an uninitialized collection error if `t_employees` is not initialized.

```
1  declare
2      type t_employee_type is table of employees.employee_id%type;
3      t_employees t_employee_type := t_employee_type();
4  begin
5      <<process_employees>>
6      for i in 1..t_employees.count()
7      loop
8          sys.dbms_output.put_line(t_employees(i)); -- some processing
9      end loop process_employees;
10 end;
11 /
```

EXAMPLE (GOOD)

Raises neither an error nor checking whether the array is empty. `t_employees.count()` always returns a `number` (unless the array is not initialized). If the array is empty `count()` returns 0 and therefore the loop will not be entered.

```
1  declare
2      type t_employee_type is table of employees.employee_id%type;
3      t_employees t_employee_type := t_employee_type();
4  begin
5      if t_employees is not null then
6          <<process_employees>>
7          for i in 1..t_employees.count()
8          loop
9              sys.dbms_output.put_line(t_employees(i)); -- some processing
10         end loop process_employees;
11     end if;
12 end;
13 /
```

G-4360: Always use a WHILE loop to process a loose array.

Minor

Efficiency

REASON

When a loose (also called sparse) array is processed using a *numeric for loop* we have to check with all iterations whether the element exist to avoid a *no_data_found* exception. In addition, the number of iterations is not driven by the number of elements in the array but by the number of the lowest/highest element. The more gaps we have, the more superfluous iterations will be done.

EXAMPLE (BAD)

```
1  declare -- raises no_data_found when processing 2nd record
2      type t_employee_type is table of employees.employee_id%type;
3      t_employees t_employee_type;
4      co_rogers      constant integer := 134;
5      co_matos       constant integer := 143;
6      co_mcewen      constant integer := 158;
7      co_index_matos constant integer := 2;
8  begin
9      t_employees := t_employee_type(co_rogers, co_matos, co_mcewen);
10     t_employees.delete(co_index_matos);
11
12     if t_employees is not null then
13         <<process_employees>>
14         for i in 1..t_employees.count()
15             loop
16                 sys.dbms_output.put_line(t_employees(i));
17             end loop process_employees;
18     end if;
19 end;
20 /
```

EXAMPLE (GOOD)

```
1  declare
2      type t_employee_type is table of employees.employee_id%type;
3      t_employees t_employee_type;
4      co_rogers      constant integer := 134;
5      co_matos       constant integer := 143;
6      co_mcewen      constant integer := 158;
7      co_index_matos constant integer := 2;
8      l_index        pls_integer;
9  begin
10     t_employees := t_employee_type(co_rogers, co_matos, co_mcewen);
11     t_employees.delete(co_index_matos);
12
13     l_index := t_employees.first();
14
15     <<process_employees>>
16     while l_index is not null
17         loop
18             sys.dbms_output.put_line(t_employees(l_index));
19             l_index := t_employees.next(l_index);
20         end loop process_employees;
21 end;
22 /
```


G-4370: Avoid using EXIT to stop loop processing unless you are in a basic loop.

Major

Maintainability

REASON

A numeric for loop as well as a while loop and a cursor for loop have defined loop boundaries. If you are not able to exit your loop using those loop boundaries, then a basic loop is the right loop to choose.

EXAMPLE (BAD)

```
1  declare
2      i integer;
3      co_min_value constant simple_integer := 1;
4      co_max_value constant simple_integer := 10;
5      co_increment constant simple_integer := 1;
6  begin
7      i := co_min_value;
8      <<while_loop>>
9      while (i <= co_max_value)
10     loop
11         i := i + co_increment;
12         exit while_loop when i > co_max_value;
13     end loop while_loop;
14
15     <<basic_loop>>
16     loop
17         exit basic_loop;
18     end loop basic_loop;
19
20     <<for_loop>>
21     for i in co_min_value..co_max_value
22     loop
23         null;
24         exit for_loop when i = co_max_value;
25     end loop for_loop;
26
27     <<process_employees>>
28     for r_employee in (select last_name
29                       from employees)
30     loop
31         sys.dbms_output.put_line(r_employee.last_name);
32         null; -- some processing
33         exit process_employees;
34     end loop process_employees;
35 end;
36 /
```

EXAMPLE (GOOD)

```

1  declare
2      i integer;
3      co_min_value constant simple_integer := 1;
4      co_max_value constant simple_integer := 10;
5      co_increment constant simple_integer := 1;
6  begin
7      i := co_min_value;
8      <<while_loop>>
9      while (i <= co_max_value)
10     loop
11         i := i + co_increment;
12     end loop while_loop;
13
14     <<basic_loop>>
15     loop
16         exit basic_loop;
17     end loop basic_loop;
18
19     <<for_loop>>
20     for i in co_min_value..co_max_value
21     loop
22         sys.dbms_output.put_line(i);
23     end loop for_loop;
24
25     <<process_employees>>
26     for r_employee in (select last_name
27                        from employees)
28     loop
29         sys.dbms_output.put_line(r_employee.last_name); -- some processing
30     end loop process_employees;
31 end;
32 /

```

G-4375: Always use EXIT WHEN instead of an IF statement to exit from a loop.

Minor

Maintainability

REASON

If you need to use an `exit` statement use its full semantic to make the code easier to understand and maintain. There is simply no need for an additional `if` statement.

EXAMPLE (BAD)

```
1  declare
2      co_first_year constant pls_integer := 1900;
3  begin
4      <<process_employees>>
5      loop
6          my_package.some_processing();
7
8          if extract(year from sysdate) > co_first_year then
9              exit process_employees;
10         end if;
11
12         my_package.some_further_processing();
13     end loop process_employees;
14 end;
15 /
```

EXAMPLE (GOOD)

```
1  declare
2      co_first_year constant pls_integer := 1900;
3  begin
4      <<process_employees>>
5      loop
6          my_package.some_processing();
7
8          exit process_employees when extract(year from sysdate) > co_first_year;
9
10         my_package.some_further_processing();
11     end loop process_employees;
12 end;
13 /
```

G-4380: Try to label your EXIT WHEN statements.

Minor

Maintainability

REASON

It's a good alternative for comments, especially for nested loops to name the loop to exit.

EXAMPLE (BAD)

```
1  declare
2      co_init_loop  constant simple_integer      := 0;
3      co_increment  constant simple_integer      := 1;
4      co_exit_value constant simple_integer      := 3;
5      co_outer_text constant types_up.short_text_type := 'Outer Loop counter is ';
6      co_inner_text constant types_up.short_text_type := ' Inner Loop counter is ';
7      l_outerlp pls_integer;
8      l_innerlp pls_integer;
9  begin
10     l_outerlp := co_init_loop;
11     <<outerloop>>
12     loop
13         l_innerlp := co_init_loop;
14         l_outerlp := nvl(l_outerlp, co_init_loop) + co_increment;
15         <<innerloop>>
16         loop
17             l_innerlp := nvl(l_innerlp, co_init_loop) + co_increment;
18             sys.dbms_output.put_line(co_outer_text || l_outerlp ||
19                                     co_inner_text || l_innerlp);
20
21             exit when l_innerlp = co_exit_value;
22         end loop innerloop;
23
24         exit when l_innerlp = co_exit_value;
25     end loop outerloop;
26 end;
27 /
```

EXAMPLE (GOOD)

```

1  declare
2      co_init_loop  constant simple_integer      := 0;
3      co_increment  constant simple_integer      := 1;
4      co_exit_value constant simple_integer      := 3;
5      co_outer_text constant types_up.short_text_type := 'Outer Loop counter is ';
6      co_inner_text constant types_up.short_text_type := ' Inner Loop counter is ';
7      l_outerlp pls_integer;
8      l_innerlp pls_integer;
9  begin
10     l_outerlp := co_init_loop;
11     <<outerloop>>
12     loop
13         l_innerlp := co_init_loop;
14         l_outerlp := nvl(l_outerlp,co_init_loop) + co_increment;
15         <<innerloop>>
16         loop
17             l_innerlp := nvl(l_innerlp, co_init_loop) + co_increment;
18             sys.dbms_output.put_line(co_outer_text || l_outerlp ||
19                                     co_inner_text || l_innerlp);
20
21             exit outerloop when l_innerlp = co_exit_value;
22         end loop innerloop;
23     end loop outerloop;
24 end;
25 /

```

G-4385: Never use a cursor for loop to check whether a cursor returns data.

Major

Efficiency

REASON

You might process more data than required, which leads to bad performance.

EXAMPLE (BAD)

```
1  declare
2      l_employee_found boolean := false;
3      cursor c_employees is
4          select employee_id, last_name
5              from employees;
6  begin
7      <<check_employees>>
8      for r_employee in c_employees
9          loop
10         l_employee_found := true;
11     end loop check_employees;
12 end;
13 /
```

EXAMPLE (GOOD)

```
1  declare
2      l_employee_found boolean := false;
3      cursor c_employees is
4          select employee_id, last_name
5              from employees;
6      r_employee c_employees%rowtype;
7  begin
8      open c_employees;
9      fetch c_employees into r_employee;
10     l_employee_found := c_employees%found;
11     close c_employees;
12 end;
13 /
```

G-4390: Avoid use of unreferenced FOR loop indexes.

Major

Efficiency

REASON

If the loop index is used for anything but traffic control inside the loop, this is one of the indicators that a numeric `for` loop is being used incorrectly. The actual body of executable statements completely ignores the loop index. When that is the case, there is a good chance that you do not need the loop at all.

EXAMPLE (BAD)

```
1  declare
2      l_row    pls_integer;
3      l_value  pls_integer;
4      co_lower_bound constant simple_integer      := 1;
5      co_upper_bound constant simple_integer      := 5;
6      co_row_incr  constant simple_integer        := 1;
7      co_value_incr constant simple_integer        := 10;
8      co_delimiter constant types_up.short_text_type := ' ';
9      co_first_value constant simple_integer      := 100;
10 begin
11     l_row := co_lower_bound;
12     l_value := co_first_value;
13     <<for_loop>>
14     for i in co_lower_bound .. co_upper_bound
15     loop
16         sys.dbms_output.put_line(l_row || co_delimiter || l_value);
17         l_row := l_row + co_row_incr;
18         l_value := l_value + co_value_incr;
19     end loop for_loop;
20 end;
21 /
```

EXAMPLE (GOOD)

```
1  declare
2      co_lower_bound constant simple_integer      := 1;
3      co_upper_bound constant simple_integer      := 5;
4      co_value_incr  constant simple_integer        := 10;
5      co_delimiter  constant types_up.short_text_type := ' ';
6      co_first_value constant simple_integer      := 100;
7  begin
8      <<for_loop>>
9      for i in co_lower_bound .. co_upper_bound
10     loop
11         sys.dbms_output.put_line(i || co_delimiter ||
12                                   to_char(co_first_value + i * co_value_incr));
13     end loop for_loop;
14 end;
15 /
```

G-4395: Avoid hard-coded upper or lower bound values with FOR loops.

Minor

Changeability, Maintainability

REASON

Your `loop` statement uses a hard-coded value for either its upper or lower bounds. This creates a "weak link" in your program because it assumes that this value will never change. A better practice is to create a named constant (or function) and reference this named element instead of the hard-coded value.

EXAMPLE (BAD)

```
1  begin
2      <<for_loop>>
3      for i in 1..5
4          loop
5              sys.dbms_output.put_line(i);
6          end loop for_loop;
7  end;
8  /
```

EXAMPLE (GOOD)

```
1  declare
2      co_lower_bound constant simple_integer := 1;
3      co_upper_bound constant simple_integer := 5;
4  begin
5      <<for_loop>>
6      for i in co_lower_bound..co_upper_bound
7          loop
8              sys.dbms_output.put_line(i);
9          end loop for_loop;
10 end;
11 /
```


Exception Handling

G-5010: Try to use a error/logging framework for your application.

Critical

Reliability, Reusability, Testability

Unsupported in PL/SQL Cop Validators

We cannot identify logging framework and where it should be applied. Requires further definition regarding naming of the error/logging framework and its minimal use in PL/SQL code.

Reason

Having a framework to raise/handle/log your errors allows you to easily avoid duplicate application error numbers and having different error messages for the same type of error.

This kind of framework should include

- Logging (different channels like table, mail, file, etc. if needed)
- Error Raising
- Multilanguage support if needed
- Translate ORACLE error messages to a user friendly error text
- Error repository

Example (bad)

```
1  begin
2      sys.dbms_output.put_line('START');
3      -- some processing
4      sys.dbms_output.put_line('END');
5  end;
6  /
```

Example (good)

```
1  declare
2      -- see https://github.com/OraOpenSource/Logger
3      l_scope logger_logs.scope%type := 'DEMO';
4  begin
5      logger.log('START', l_scope);
6      -- some processing
7      logger.log('END', l_scope);
8  end;
9  /
```

G-5020: Never handle unnamed exceptions using the error number.

Critical

Maintainability

Reason

When literals are used for error numbers the reader needs the error message manual to understand what is going on. Commenting the code or using constants is an option, but it is better to use named exceptions instead, because it ensures a certain level of consistency which makes maintenance easier.

Example (bad)

```
1  declare
2      co_no_data_found constant integer := -1;
3  begin
4      my_package.some_processing(); -- some code which raises an exception
5  exception
6      when too_many_rows then
7          my_package.some_further_processing();
8      when others then
9          if sqlcode = co_no_data_found then
10             null;
11          end if;
12  end;
13  /
```

Example (good)

```
1  begin
2      my_package.some_processing(); -- some code which raises an exception
3  exception
4      when too_many_rows then
5          my_package.some_further_processing();
6      when no_data_found then
7          null; -- handle no_data_found
8  end;
9  /
```

G-5030: Never assign predefined exception names to user defined exceptions.

Blocker

Reliability, Testability

Reason

This is error-prone because your local declaration overrides the global declaration. While it is technically possible to use the same names, it causes confusion for others needing to read and maintain this code. Additionally, you will need to be very careful to use the prefix `standard` in front of any reference that needs to use Oracle's default exception behavior.

Example (bad)

Using the code below, we are not able to handle the `no_data_found` exception raised by the `select` statement as we have overwritten that exception handler. In addition, our exception handler doesn't have an exception number assigned, which should be raised when the `select` statement does not find any rows.

```
1  declare
2      l_dummy dual.dummy%type;
3      no_data_found    exception;
4      co_rownum        constant simple_integer          := 0;
5      co_no_data_found constant types_up.short_text_type := 'no_data_found';
6  begin
7      select dummy
8          into l_dummy
9          from dual
10         where rownum = co_rownum;
11
12     if l_dummy is null then
13         raise no_data_found;
14     end if;
15 exception
16     when no_data_found then
17         sys.dbms_output.put_line(co_no_data_found);
18 end;
19 /
20
21 Error report -
22 ORA-01403: no data found
23 ORA-06512: at line 5
24 01403. 00000 - "no data found"
25 *Cause:      No data was found from the objects.
26 *Action:     There was no data from the objects which may be due to end of fetch.
```

Example (good)

```

1  declare
2      l_dummy dual.dummy%type;
3      empty_value      exception;
4      co_rownum        constant simple_integer      := 0;
5      co_empty_value    constant types_up.short_text_type := 'empty_value';
6      co_no_data_found  constant types_up.short_text_type := 'no_data_found';
7  begin
8      select dummy
9          into l_dummy
10         from dual
11        where rownum = co_rownum;
12
13     if l_dummy is null then
14         raise empty_value;
15     end if;
16 exception
17     when empty_value then
18         sys.dbms_output.put_line(co_empty_value);
19     when no_data_found then
20         sys.dbms_output.put_line(co_no_data_found);
21 end;
22 /

```

G-5040: Avoid use of WHEN OTHERS clause in an exception section without any other specific handlers.

Major

Reliability

Reason

There is not necessarily anything wrong with using `when others`, but it can cause you to "lose" error information unless your handler code is relatively sophisticated. Generally, you should use `when others` to grab any and every error only after you have thought about your executable section and decided that you are not able to trap any specific exceptions. If you know, on the other hand, that a certain exception might be raised, include a handler for that error. By declaring two different exception handlers, the code more clearly states what we expect to have happen and how we want to handle the errors. That makes it easier to maintain and enhance. We also avoid hard-coding error numbers in checks against `sqlcode`.

When using a logging framework like Logger, consider making an exception to this rule and allow a `when others` even without other specific handlers, but *only* if the `when others` exception handler calls a logging procedure that saves the error stack (that otherwise is lost) and the last statement of the handler is `raise`.

Example (bad)

```
1  begin
2      my_package.some_processing();
3  exception
4      when others then
5          my_package.some_further_processing();
6  end;
7  /
```

Example (good)

```
1  begin
2      my_package.some_processing();
3  exception
4      when dup_val_on_index then
5          my_package.some_further_processing();
6  end;
7  /
```

Example (exception to the rule)

```
1  begin
2      my_package.some_processing();
3  exception
4      when others then
5          logger.log_error('Unhandled Exception');
6          raise;
7  end;
8  /
```

G-5050: Avoid use of the RAISE_APPLICATION_ERROR built-in procedure with a hard-coded 20nnn error number or hard-coded message.

Major

Changeability, Maintainability

Reason

If you are not very organized in the way you allocate, define and use the error numbers between 20999 and 20000 (those reserved by Oracle for its user community), it is very easy to end up with conflicting usages. You should assign these error numbers to named constants and consolidate all definitions within a single package. When you call `raise_application_error`, you should reference these named elements and error message text stored in a table. Use your own raise procedure in place of explicit calls to `raise_application_error`. If you are raising a "system" exception like `no_data_found`, you must use `raise`. However, when you want to raise an application-specific error, you use `raise_application_error`. If you use the latter, you then have to provide an error number and message. This leads to unnecessary and damaging hard-coded values. A more fail-safe approach is to provide a predefined raise procedure that automatically checks the error number and determines the correct way to raise the error.

Example (bad)

```
1  begin
2      raise_application_error(-20501, 'Invalid employee_id');
3  end;
4  /
```

Example (good)

```
1  begin
2      err_up.raise(in_error => err.co_invalid_employee_id);
3  end;
4  /
```

G-5060: Avoid unhandled exceptions.

Major

Reliability

Reason

This may be your intention, but you should review the code to confirm this behavior.

If you are raising an error in a program, then you are clearly predicting a situation in which that error will occur. You should consider including a handler in your code for predictable errors, allowing for a graceful and informative failure. After all, it is much more difficult for an enclosing block to be aware of the various errors you might raise and more importantly, what should be done in response to the error.

The form that this failure takes does not necessarily need to be an exception. When writing functions, you may well decide that in the case of certain exceptions, you will want to return a value such as `null`, rather than allow an exception to propagate out of the function.

Example (bad)

```
1  create or replace package body department_api is
2      function name_by_id (in_id in departments.department_id%type)
3          return departments.department_name%type is
4          l_department_name departments.department_name%type;
5      begin
6          select department_name
7              into l_department_name
8              from departments
9              where department_id = in_id;
10
11          return l_department_name;
12      end name_by_id;
13 end department_api;
14 /
```

Example (good)

```
1  create or replace package body department_api is
2      function name_by_id (in_id in departments.department_id%type)
3          return departments.department_name%type is
4          l_department_name departments.department_name%type;
5      begin
6          select department_name
7              into l_department_name
8              from departments
9              where department_id = in_id;
10
11          return l_department_name;
12      exception
13          when no_data_found then return null;
14          when too_many_rows then raise;
15      end name_by_id;
16 end department_api;
17 /
```

G-5070: Avoid using Oracle predefined exceptions.

Critical

Reliability

Reason

You have raised an exception whose name was defined by Oracle. While it is possible that you have a good reason for "using" one of Oracle's predefined exceptions, you should make sure that you would not be better off declaring your own exception and raising that instead.

If you decide to change the exception you are using, you should apply the same consideration to your own exceptions. Specifically, do not "re-use" exceptions. You should define a separate exception for each error condition, rather than use the same exception for different circumstances.

Being as specific as possible with the errors raised will allow developers to check for, and handle, the different kinds of errors the code might produce.

Example (bad)

```
1  begin
2      raise no_data_found;
3  end;
4  /
```

Example (good)

```
1  declare
2      my_exception exception;
3  begin
4      raise my_exception;
5  end;
6  /
```


G-5080: Always use FORMAT_ERROR_BACKTRACE when using FORMAT_ERROR_STACK or SQLERRM.

Minor

Maintainability, Testability

Reason

In exception handler `sqlerrm` and `format_error_stack` won't tell you the exact line where the error occurred.

`format_error_backtrace` displays the call stack at the point where an exception was raised, even if the subprogram is called from an exception handler in an outer scope.

If you use `sqlerrm` or `format_error_stack` to log/display error, you should also include `format_error_backtrace` to identify the exact location where the exception was raised.

Example (bad)

```
1  create or replace package body order_api as
2      procedure discount_and_recalculate(
3          in_customer_id    customer.id%type
4          , in_discount      customer.discount_percentage%type
5      )
6      begin
7          customer_api.apply_discount(in_customer_id, in_discount);
8          customer_api.in_customer_id(10293847);
9      exception
10         when zero_divide then
11             null; -- ignore
12         when others then
13             logging_package.log_error('Error: ' || sqlerrm);
14             raise;
15     end discount_and_recalculate;
16 end order_api;
17 /
```

Example (good)

```
1  create or replace package body order_api as
2      procedure discount_and_recalculate(
3          in_customer_id    customer.id%type
4          , in_discount      customer.discount_percentage%type
5      )
6      begin
7          customer_api.apply_discount(in_customer_id, in_discount);
8          customer_api.in_customer_id(10293847);
9      exception
10         when zero_divide then
11             null; -- ignore
12         when others then
13             logging_package.log_error(
14                 'Error: ' || sqlerrm || ' - Backtrace: ' || dbms_utility.format_error_backtrace
15             );
16             raise;
17     end discount_and_recalculate;
18 end order_api;
19 /
```

Dynamic SQL

G-6010: Always use a character variable to execute dynamic SQL.

Major

Maintainability, Testability

Reason

Having the executed statement in a variable makes it easier to debug your code (e.g. by logging the statement that failed).

Example (bad)

```
1 declare
2     l_next_val employees.employee_id%type;
3 begin
4     execute immediate 'select employees_seq.nextval from dual' into l_next_val;
5 end;
6 /
```

Example (good)

```
1 declare
2     l_next_val employees.employee_id%type;
3     co_sql constant types_up.big_string_type :=
4         'select employees_seq.nextval from dual';
5 begin
6     execute immediate co_sql into l_next_val;
7 end;
8 /
```

G-6020: Try to use output bind arguments in the RETURNING INTO clause of dynamic DML statements rather than the USING clause.

Minor

Maintainability

Reason

When a dynamic `insert`, `update`, or `delete` statement has a `returning` clause, output bind arguments can go in the `returning into` clause or in the `using` clause.

You should use the `returning into` clause for values returned from a DML operation. Reserve `out` and `in out` bind variables for dynamic PL/SQL blocks that return values in PL/SQL variables.

Example (bad)

```
1  create or replace package body employee_api is
2      procedure upd_salary (in_employee_id in      employees.employee_id%type
3                          ,in_increase_pct in      types_up.percentage
4                          ,out_new_salary out      employees.salary%type)
5  is
6      co_sql_stmt constant types_up.big_string_type := '
7          update employees set salary = salary + (salary / 100 * :1)
8          where employee_id = :2
9          returning salary into :3';
10  begin
11      execute immediate co_sql_stmt
12          using in_increase_pct, in_employee_id, out out_new_salary;
13  end upd_salary;
14  end employee_api;
15  /
```

Example (good)

```
1  create or replace package body employee_api is
2      procedure upd_salary (in_employee_id in      employees.employee_id%type
3                          ,in_increase_pct in      types_up.percentage
4                          ,out_new_salary out      employees.salary%type)
5  is
6      co_sql_stmt constant types_up.big_string_type :=
7          'update employees set salary = salary + (salary / 100 * :1)
8          where employee_id = :2
9          returning salary into :3';
10  begin
11      execute immediate co_sql_stmt
12          using in_increase_pct, in_employee_id
13          returning into out_new_salary;
14  end upd_salary;
15  end employee_api;
16  /
```

Stored Objects

General

G-7110: Try to use named notation when calling program units.

Major

Changeability, Maintainability

REASON

Named notation makes sure that changes to the signature of the called program unit do not affect your call.

This is not needed for standard functions like (`to_char`, `to_date`, `nvl`, `round`, etc.) but should be followed for any other stored object having more than one parameter.

EXAMPLE (BAD)

```
1  declare
2      r_employee employees%rowtype;
3      co_id constant employees.employee_id%type := 107;
4  begin
5      employee_api.employee_by_id(r_employee, co_id);
6  end;
7  /
```

EXAMPLE (GOOD)

```
1  declare
2      r_employee employees%rowtype;
3      co_id constant employees.employee_id%type := 107;
4  begin
5      employee_api.employee_by_id(out_row => r_employee, in_employee_id => co_id);
6  end;
7  /
```

G-7120: Always add the name of the program unit to its end keyword.

Minor

Maintainability

REASON

It's a good alternative for comments to indicate the end of program units, especially if they are lengthy or nested.

EXAMPLE (BAD)

```
1  create or replace package body employee_api is
2      function employee_by_id (in_employee_id in employees.employee_id%type)
3          return employees%rowtype is
4          r_employee employees%rowtype;
5      begin
6          select *
7              into r_employee
8              from employees
9              where employee_id = in_employee_id;
10
11         return r_employee;
12     exception
13         when no_data_found then
14             null;
15         when too_many_rows then
16             raise;
17     end;
18 end;
19 /
```

EXAMPLE (GOOD)

```
1  create or replace package body employee_api is
2      function employee_by_id (in_employee_id in employees.employee_id%type)
3          return employees%rowtype is
4          r_employee employees%rowtype;
5      begin
6          select *
7              into r_employee
8              from employees
9              where employee_id = in_employee_id;
10
11         return r_employee;
12     exception
13         when no_data_found then
14             null;
15         when too_many_rows then
16             raise;
17     end employee_by_id;
18 end employee_api;
19 /
```

G-7125: Always use CREATE OR REPLACE instead of CREATE alone.

Minor

Maintainability

REASON

Using `create` alone makes your scripts give an error if the program unit already exists, which makes the script not repeatable. It is good practice to use `create or replace` to avoid such errors.

EXAMPLE (BAD)

```
1  create package body employee_api is
2      function employee_by_id (in_employee_id in employees.employee_id%type)
3          return employees%rowtype is
4          r_employee employees%rowtype;
5      begin
6          select *
7              into r_employee
8              from employees
9              where employee_id = in_employee_id;
10
11         return r_employee;
12     exception
13         when no_data_found then
14             null;
15         when too_many_rows then
16             raise;
17     end employee_by_id;
18 end employee_api;
19 /
```

EXAMPLE (GOOD)

```
1  create or replace package body employee_api is
2      function employee_by_id (in_employee_id in employees.employee_id%type)
3          return employees%rowtype is
4          r_employee employees%rowtype;
5      begin
6          select *
7              into r_employee
8              from employees
9              where employee_id = in_employee_id;
10
11         return r_employee;
12     exception
13         when no_data_found then
14             null;
15         when too_many_rows then
16             raise;
17     end employee_by_id;
18 end employee_api;
19 /
```

G-7130: Always use parameters or pull in definitions rather than referencing external variables in a local program unit.

Major

Maintainability, Reliability, Testability

REASON

Local procedures and functions offer an excellent way to avoid code redundancy and make your code more readable (and thus more maintainable). Your local program refers, however, an external data structure, i.e., a variable that is declared outside of the local program. Thus, it is acting as a global variable inside the program.

This external dependency is hidden, and may cause problems in the future. You should instead add a parameter to the parameter list of this program and pass the value through the list. This technique makes your program more reusable and avoids scoping problems, i.e. the program unit is less tied to particular variables in the program. In addition, unit encapsulation makes maintenance a lot easier and cheaper.

EXAMPLE (BAD)

```
1  create or replace package body employee_api is
2      procedure calc_salary (in_employee_id in employees.employee_id%type) is
3          r_emp employees%rowtype;
4
5          function commission return number is
6              l_commission employees.salary%type := 0;
7          begin
8              if r_emp.commission_pct is not null
9              then
10                 l_commission := r_emp.salary * r_emp.commission_pct;
11             end if;
12
13             return l_commission;
14         end commission;
15     begin
16         select *
17         into r_emp
18         from employees
19         where employee_id = in_employee_id;
20
21         sys.dbms_output.put_line(r_emp.salary + commission());
22     exception
23         when no_data_found then
24             null;
25         when too_many_rows then
26             null;
27     end calc_salary;
28 end employee_api;
29 /
```

EXAMPLE (GOOD)

```

1  create or replace package body employee_api is
2      procedure calc_salary (in_employee_id in employees.employee_id%type) is
3          r_emp employees%rowtype;
4
5          function commission (in_salary    in employees.salary%type
6                               ,in_comm_pct in employees.commission_pct%type)
7              return number is
8              l_commission employees.salary%type := 0;
9      begin
10         if in_comm_pct is not null then
11             l_commission := in_salary * in_comm_pct;
12         end if;
13
14         return l_commission;
15     end commission;
16 begin
17     select *
18     into r_emp
19     from employees
20     where employee_id = in_employee_id;
21
22     sys.dbms_output.put_line(
23         r_emp.salary + commission(in_salary => r_emp.salary
24                                   ,in_comm_pct => r_emp.commission_pct)
25     );
26 exception
27     when no_data_found then
28         null;
29     when too_many_rows then
30         null;
31 end calc_salary;
32 end employee_api;
33 /

```


G-7140: Always ensure that locally defined procedures or functions are referenced.

Major

Maintainability, Reliability

REASON

This can occur as the result of changes to code over time, but you should make sure that this situation does not reflect a problem. And you should remove the declaration to avoid maintenance errors in the future.

You should go through your programs and remove any part of your code that is no longer used. This is a relatively straightforward process for variables and named constants. Simply execute searches for a variable's name in that variable's scope. If you find that the only place it appears is in its declaration, delete the declaration.

There is never a better time to review all the steps you took, and to understand the reasons you took them, than immediately upon completion of your program. If you wait, you will find it particularly difficult to remember those parts of the program that were needed at one point, but were rendered unnecessary in the end.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure my_procedure is
3          function my_func return number is
4              co_true constant integer := 1;
5          begin
6              return co_true;
7          end my_func;
8      begin
9          null;
10     end my_procedure;
11 end my_package;
12 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      procedure my_procedure is
3          function my_func return number is
4              co_true constant integer := 1;
5          begin
6              return co_true;
7          end my_func;
8      begin
9          sys.dbms_output.put_line(my_func());
10     end my_procedure;
11 end my_package;
12 /
```

G-7150: Try to remove unused parameters.

Minor

Efficiency, Maintainability

REASON

You should go through your programs and remove any parameter that is no longer used.

EXAMPLE (BAD)

```
1  create or replace package body department_api is
2      function name_by_id (in_department_id in departments.department_id%type
3                          ,in_manager_id   in departments.manager_id%type)
4          return departments.department_name%type is
5      l_department_name departments.department_name%type;
6  begin
7      <<find_department>>
8      begin
9          select department_name
10             into l_department_name
11             from departments
12             where department_id = in_department_id;
13      exception
14          when no_data_found or too_many_rows then
15              l_department_name := null;
16      end find_department;
17
18      return l_department_name;
19  end name_by_id;
20 end department_api;
21 /
```

EXAMPLE (GOOD)

```
1  create or replace package body department_api is
2      function name_by_id (in_department_id in departments.department_id%type)
3          return departments.department_name%type is
4      l_department_name departments.department_name%type;
5  begin
6      <<find_department>>
7      begin
8          select department_name
9             into l_department_name
10             from departments
11             where department_id = in_department_id;
12      exception
13          when no_data_found or too_many_rows then
14              l_department_name := null;
15      end find_department;
16
17      return l_department_name;
18  end name_by_id;
19 end department_api;
20 /
```

 Major

Maintainability

REASON

By showing the mode of parameters, you help the reader. If you do not specify a parameter mode, the default mode is `in`. Explicitly showing the mode indication of all parameters is a more assertive action than simply taking the default mode. Anyone reviewing the code later will be more confident that you intended the parameter mode to be `in`, `out` or `in out`.

EXAMPLE (BAD)

```
1  create or replace package employee_api is
2      procedure upsert (io_id          in out employees.id%type
3                        ,in_first_name  employees.first_name%type
4                        ,in_last_name   employees.last_name%type
5                        ,in_email       employees.email%type
6                        ,in_department_id employees.department_id%type
7                        ,out_success    out pls_integer);
8  end employee_up;
9  /
```

EXAMPLE (GOOD)

```
1  create or replace package employee_api is
2      procedure upsert (io_id          in out employees.id%type
3                        ,in_first_name  in  employees.first_name%type
4                        ,in_last_name   in  employees.last_name%type
5                        ,in_email       in  employees.email%type
6                        ,in_department_id in  employees.department_id%type
7                        ,out_success    out pls_integer);
8  end employee_up;
9  /
```

G-7170: Avoid using an IN OUT parameter as IN or OUT only.

Major

Efficiency, Maintainability

Unsupported in PL/SQL Cop Validators

We cannot determine the usage of an `in out` parameter in a reliable way, especially when other units are involved which are maintained in another file.

REASON

Avoid using parameter mode `in out` unless you actually use the parameter both as input and output. If the code body only reads from the parameter, use `in`; if the code body only assigns to the parameter, use `out`. If at the beginning of a project you expect a parameter to be both input and output and therefore choose `in out` just in case, but later development shows the parameter actually is only `in` or `out`, you should change the parameter mode accordingly.

EXAMPLE (BAD)

```
1  create or replace package body employee_up is
2      procedure rcv_emp (io_first_name    in out employees.first_name%type
3                        ,io_last_name     in out employees.last_name%type
4                        ,io_email         in out employees.email%type
5                        ,io_phone_number  in out employees.phone_number%type
6                        ,io_hire_date     in out employees.hire_date%type
7                        ,io_job_id        in out employees.job_id%type
8                        ,io_salary       in out employees.salary%type
9                        ,io_commission_pct in out employees.commission_pct%type
10                       ,io_manager_id    in out employees.manager_id%type
11                       ,io_department_id in out employees.department_id%type
12                       ,in_wait          in    integer) is
13      l_status pls_integer;
14      co_dflt_pipe_name constant string(30 char) := 'MyPipe';
15      co_ok constant pls_integer := 1;
16  begin
17      -- Receive next message and unpack for each column.
18      l_status := sys.dbms_pipe.receive_message(pipename => co_dflt_pipe_name
19                                              ,timeout => in_wait);
20      if l_status = co_ok then
21          sys.dbms_pipe.unpack_message (io_first_name);
22          sys.dbms_pipe.unpack_message (io_last_name);
23          sys.dbms_pipe.unpack_message (io_email);
24          sys.dbms_pipe.unpack_message (io_phone_number);
25          sys.dbms_pipe.unpack_message (io_hire_date);
26          sys.dbms_pipe.unpack_message (io_job_id);
27          sys.dbms_pipe.unpack_message (io_salary);
28          sys.dbms_pipe.unpack_message (io_commission_pct);
29          sys.dbms_pipe.unpack_message (io_manager_id);
30          sys.dbms_pipe.unpack_message (io_department_id);
31      end if;
32  end rcv_emp;
33 end employee_up;
34 /
```

EXAMPLE (GOOD)

```

1  create or replace package body employee_up is
2      procedure rcv_emp (out_first_name      out employees.first_name%type
3                          ,out_last_name      out employees.last_name%type
4                          ,out_email          out employees.email%type
5                          ,out_phone_number    out employees.phone_number%type
6                          ,out_hire_date       out employees.hire_date%type
7                          ,out_job_id          out employees.job_id%type
8                          ,out_salary          out employees.salary%type
9                          ,out_commission_pct  out employees.commission_pct%type
10                         ,out_manager_id      out employees.manager_id%type
11                         ,out_department_id   out employees.department_id%type
12                         ,in_wait            in integer) is
13      l_status pls_integer;
14      co_dflt_pipe_name constant string(30 char) := 'MyPipe';
15      co_ok constant pls_integer := 1;
16  begin
17      -- Receive next message and unpack for each column.
18      l_status := sys.dbms_pipe.receive_message(pipename => co_dflt_pipe_name
19                                              ,timeout => in_wait);
20      if l_status = co_ok then
21          sys.dbms_pipe.unpack_message (out_first_name);
22          sys.dbms_pipe.unpack_message (out_last_name);
23          sys.dbms_pipe.unpack_message (out_email);
24          sys.dbms_pipe.unpack_message (out_phone_number);
25          sys.dbms_pipe.unpack_message (out_hire_date);
26          sys.dbms_pipe.unpack_message (out_job_id);
27          sys.dbms_pipe.unpack_message (out_salary);
28          sys.dbms_pipe.unpack_message (out_commission_pct);
29          sys.dbms_pipe.unpack_message (out_manager_id);
30          sys.dbms_pipe.unpack_message (out_department_id);
31      end if;
32  end rcv_emp;
33 end employee_up;
34 /

```

Packages

G-7210: Try to keep your packages small. Include only few procedures and functions that are used in the same context.

Minor

Efficiency, Maintainability

REASON

The entire package is loaded into memory when the package is called the first time. To optimize memory consumption and keep load time small packages should be kept small but include components that are used together.

G-7220: Always use forward declaration for private functions and procedures.

Minor

Changeability

REASON

Having forward declarations allows you to order the functions and procedures of the package in a reasonable way.

EXAMPLE (BAD)

```
1  create or replace package department_api is
2      procedure del (in_department_id in departments.department_id%type);
3  end department_api;
4  /
5
6  create or replace package body department_api is
7      function does_exist (in_department_id in departments.department_id%type)
8          return boolean is
9          l_return pls_integer;
10     begin
11         <<check_row_exists>>
12         begin
13             select 1
14                 into l_return
15                 from departments
16                 where department_id = in_department_id;
17         exception
18             when no_data_found or too_many_rows then
19                 l_return := 0;
20         end check_row_exists;
21
22         return l_return = 1;
23     end does_exist;
24
25     procedure del (in_department_id in departments.department_id%type) is
26     begin
27         if does_exist(in_department_id) then
28             null;
29         end if;
30     end del;
31 end department_api;
32 /
```

EXAMPLE (GOOD)

```

1  create or replace package department_api is
2      procedure del (in_department_id in departments.department_id%type);
3  end department_api;
4  /
5
6  create or replace package body department_api is
7      function does_exist (in_department_id in departments.department_id%type)
8          return boolean;
9
10     procedure del (in_department_id in departments.department_id%type) is
11     begin
12         if does_exist(in_department_id) then
13             null;
14         end if;
15     end del;
16
17     function does_exist (in_department_id in departments.department_id%type)
18         return boolean is
19         l_return pls_integer;
20     begin
21         <<check_row_exists>>
22         begin
23             select 1
24                 into l_return
25                 from departments
26                 where department_id = in_department_id;
27         exception
28             when no_data_found or too_many_rows then
29                 l_return := 0;
30         end check_row_exists;
31
32         return l_return = 1;
33     end does_exist;
34 end department_api;
35 /

```


G-7230: Avoid declaring global variables public.

Major

Reliability

REASON

You should always declare package-level data (non-constants) inside the package body. You can then define "get and set" methods (functions and procedures, respectively) in the package specification to provide controlled access to that data. By doing so you can guarantee data integrity, you can change your data structure implementation, and also track access to those data structures.

Data structures (scalar variables, collections, cursors) declared in the package specification (not within any specific program) can be referenced directly by any program running in a session with `execute` rights to the package.

Instead, declare all package-level data in the package body and provide "get and set" methods - a function to get the value and a procedure to set the value - in the package specification. Developers then can access the data using these methods - and will automatically follow all rules you set upon data modification.

For package-level constants, consider whether the constant should be public and usable from other code, or if only relevant for code within the package. If the latter, declare the constant in the package body. If the former, it is typically good practice to place the constants in a package specification that only holds constants.

EXAMPLE (BAD)

```
1  create or replace package employee_api as
2      co_min_increase constant types_up.sal_increase_type := 0.01;
3      co_max_increase constant types_up.sal_increase_type := 0.5;
4      g_salary_increase types_up.sal_increase_type := co_min_increase;
5
6      procedure set_salary_increase (in_increase in types_up.sal_increase_type);
7      function salary_increase return types_up.sal_increase_type;
8  end employee_api;
9  /
10
11 create or replace package body employee_api as
12     procedure set_salary_increase (in_increase in types_up.sal_increase_type) is
13     begin
14         g_salary_increase := greatest(least(in_increase,co_max_increase)
15                                     ,co_min_increase);
16     end set_salary_increase;
17
18     function salary_increase return types_up.sal_increase_type is
19     begin
20         return g_salary_increase;
21     end salary_increase;
22 end employee_api;
23 /
```

EXAMPLE (GOOD)

```

1  create or replace package constants_up as
2      co_min_increase constant types_up.sal_increase_type := 0.01;
3      co_max_increase constant types_up.sal_increase_type := 0.5;
4  end constants_up;
5  /
6
7  create or replace package employee_api as
8      procedure set_salary_increase (in_increase in types_up.sal_increase_type);
9      function salary_increase return types_up.sal_increase_type;
10 end employee_api;
11 /
12
13 create or replace package body employee_api as
14     g_salary_increase types_up.sal_increase_type(4,2);
15
16     procedure init;
17
18     procedure set_salary_increase (in_increase in types_up.sal_increase_type) is
19     begin
20         g_salary_increase := greatest(least(in_increase
21                                     , constants_up.co_max_increase)
22                                     , constants_up.co_min_increase);
23     end set_salary_increase;
24
25     function salary_increase return types_up.sal_increase_type is
26     begin
27         return g_salary_increase;
28     end salary_increase;
29
30     procedure init
31     is
32     begin
33         g_salary_increase := constants_up.co_min_increase;
34     end init;
35 begin
36     init();
37 end employee_api;
38 /

```

G-7250: Never use RETURN in package initialization block.

Minor

Maintainability

REASON

The purpose of the initialization block of a package body is to set initial values of the global variables of the package (initialize the package state). Although `return` is syntactically allowed in this block, it makes no sense. If it is the last keyword of the block, it is superfluous. If it is not the last keyword, then all code after the `return` is unreachable and thus dead code.

EXAMPLE (BAD)

```
1  create or replace package body employee_api as
2      g_salary_increase types_up.sal_increase_type(4,2);
3
4      procedure set_salary_increase (in_increase in types_up.sal_increase_type) is
5      begin
6          g_salary_increase := greatest(least(in_increase
7                                          , constants_up.max_salary_increase())
8                                          , constants_up.min_salary_increase());
9      end set_salary_increase;
10
11     function salary_increase return types_up.sal_increase_type is
12     begin
13         return g_salary_increase;
14     end salary_increase;
15
16     begin
17         g_salary_increase := constants_up.min_salary_increase();
18
19         return;
20
21         set_salary_increase(constants_up.min_salary_increase()); -- dead code
22     end employee_api;
23 /
```

EXAMPLE (GOOD)

```
1  create or replace package body employee_api as
2      g_salary_increase types_up.sal_increase_type(4,2);
3
4      procedure set_salary_increase (in_increase in types_up.sal_increase_type) is
5      begin
6          g_salary_increase := greatest(least(in_increase
7                                          , constants_up.max_salary_increase())
8                                          , constants_up.min_salary_increase());
9      end set_salary_increase;
10
11     function salary_increase return types_up.sal_increase_type is
12     begin
13         return g_salary_increase;
14     end salary_increase;
15
16     begin
17         g_salary_increase := constants_up.min_salary_increase();
18     end employee_api;
19 /
```

Procedures

G-7310: Avoid standalone procedures – put your procedures in packages.

 **Minor**

Maintainability

REASON

Use packages to structure your code, combine procedures and functions which belong together.

Package bodies may be changed and compiled without invalidating other packages. This is major advantage compared to standalone procedures and functions.

EXAMPLE (BAD)

```
1  create or replace procedure my_procedure is
2  begin
3      null;
4  end my_procedure;
5  /
```

EXAMPLE (GOOD)

```
1  create or replace package my_package is
2      procedure my_procedure;
3  end my_package;
4  /
5
6  create or replace package body my_package is
7      procedure my_procedure is
8      begin
9          null;
10         end my_procedure;
11 end my_package;
12 /
```

G-7320: Avoid using RETURN statements in a PROCEDURE.

Major

Maintainability, Testability

REASON

Use of the `return` statement is legal within a procedure in PL/SQL, but it is very similar to a `goto`, which means you end up with poorly structured code that is hard to debug and maintain.

A good general rule to follow as you write your PL/SQL programs is "one way in and one way out". In other words, there should be just one way to enter or call a program, and there should be one way out, one exit path from a program (or loop) on successful termination. By following this rule, you end up with code that is much easier to trace, debug, and maintain.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure my_procedure is
3          l_idx simple_integer := 1;
4          co_modulo constant simple_integer := 7;
5      begin
6          <<mod7_loop>>
7          loop
8              if mod(l_idx,co_modulo) = 0 then
9                  return;
10             end if;
11
12             l_idx := l_idx + 1;
13         end loop mod7_loop;
14     end my_procedure;
15 end my_package;
16 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      procedure my_procedure is
3          l_idx simple_integer := 1;
4          co_modulo constant simple_integer := 7;
5      begin
6          <<mod7_loop>>
7          loop
8              exit mod7_loop when mod(l_idx,co_modulo) = 0;
9
10             l_idx := l_idx + 1;
11         end loop mod7_loop;
12     end my_procedure;
13 end my_package;
14 /
```

G-7330: Always assign values to OUT parameters.

Major

Maintainability, Testability

REASON

Marking a parameter for output means that callers will expect its value to be updated with a result from the execution of the procedure. Failing to update the parameter before the procedure returns is surely an error.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      procedure greet(
3          in_name      in  varchar2
4          , out_greeting out varchar2
5      ) is
6          l_message varchar2(100 char);
7      begin
8          l_message := 'Hello, ' || in_name;
9      end my_procedure;
10 end my_package;
11 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      procedure greet(
3          in_name      in  varchar2
4          , out_greeting out varchar2
5      ) is
6      begin
7          out_greeting := 'Hello, ' || in_name;
8      end my_procedure;
9  end my_package;
10 /
```

Functions

G-7410: Avoid standalone functions – put your functions in packages.

 **Minor**

Maintainability

REASON

Use packages to structure your code, combine procedures and functions which belong together.

Package bodies may be changed and compiled without invalidating other packages. This is major advantage compared to standalone procedures and functions.

EXAMPLE (BAD)

```
1  create or replace function my_function return varchar2 is
2  begin
3      return null;
4  end my_function;
5  /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      function my_function return varchar2 is
3      begin
4          return null;
5      end my_function;
6  end my_package;
7  /
```

G-7420: Always make the RETURN statement the last statement of your function.

Major

Maintainability

REASON

The reader expects the `return` statement to be the last statement of a function.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      function my_function (in_from in pls_integer
3                          , in_to   in pls_integer) return pls_integer is
4          l_ret pls_integer;
5      begin
6          l_ret := in_from;
7          <<for_loop>>
8          for i in in_from .. in_to
9              loop
10                 l_ret := l_ret + i;
11                 if i = in_to then
12                     return l_ret;
13                 end if;
14             end loop for_loop;
15     end my_function;
16 end my_package;
17 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      function my_function (in_from in pls_integer
3                          , in_to   in pls_integer) return pls_integer is
4          l_ret pls_integer;
5      begin
6          l_ret := in_from;
7          <<for_loop>>
8          for i in in_from .. in_to
9              loop
10                 l_ret := l_ret + i;
11             end loop for_loop;
12         return l_ret;
13     end my_function;
14 end my_package;
15 /
```


G-7430: Try to use no more than one RETURN statement within a function.

Major

Maintainability, Testability

REASON

A function should have a single point of entry as well as a single exit-point.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      function my_function (in_value in pls_integer) return boolean is
3          co_yes constant pls_integer := 1;
4      begin
5          if in_value = co_yes then
6              return true;
7          else
8              return false;
9          end if;
10     end my_function;
11 end my_package;
12 /
```

EXAMPLE (BETTER)

```
1  create or replace package body my_package is
2      function my_function (in_value in pls_integer) return boolean is
3          co_yes constant pls_integer := 1;
4          l_ret boolean;
5      begin
6          if in_value = co_yes then
7              l_ret := true;
8          else
9              l_ret := false;
10         end if;
11
12         return l_ret;
13     end my_function;
14 end my_package;
15 /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      function my_function (in_value in pls_integer) return boolean is
3          co_yes constant pls_integer := 1;
4      begin
5          return in_value = co_yes;
6      end my_function;
7 end my_package;
8 /
```

G-7440: Never use OUT parameters to return values from a function.

Major

Reusability

REASON

A function should return all its data through the `return` clause. Having an `out` parameter prohibits usage of a function within SQL statements.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      function my_function (out_date out date) return boolean is
3      begin
4          out_date := sysdate;
5          return true;
6      end my_function;
7  end my_package;
8  /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      function my_function return date is
3      begin
4          return sysdate;
5      end my_function;
6  end my_package;
7  /
```

G-7450: Never return a NULL value from a BOOLEAN function.

Major

Reliability, Testability

REASON

If a boolean function returns `null`, the caller has to deal with it. This makes the usage cumbersome and more error-prone.

EXAMPLE (BAD)

```
1  create or replace package body my_package is
2      function my_function return boolean is
3      begin
4          return null;
5      end my_function;
6  end my_package;
7  /
```

EXAMPLE (GOOD)

```
1  create or replace package body my_package is
2      function my_function return boolean is
3      begin
4          return true;
5      end my_function;
6  end my_package;
7  /
```

G-7460: Try to define your packaged/standalone function deterministic if appropriate.

Major

Efficiency

REASON

A deterministic function (always return same result for identical parameters) which is defined to be deterministic will be executed once per different parameter within a SQL statement whereas if the function is not defined to be deterministic it is executed once per result row.

EXAMPLE (BAD)

```
1  create or replace package department_api is
2      function name_by_id (in_department_id in departments.department_id%type)
3          return departments.department_name%type;
4  end department_api;
5  /
```

EXAMPLE (GOOD)

```
1  create or replace package department_api is
2      function name_by_id (in_department_id in departments.department_id%type)
3          return departments.department_name%type deterministic;
4  end department_api;
5  /
```

Oracle Supplied Packages

G-7510: Always prefix ORACLE supplied packages with owner schema name.

Major

Security

REASON

The signature of oracle-supplied packages is well known and therefore it is quite easy to provide packages with the same name as those from oracle doing something completely different without you noticing it.

EXAMPLE (BAD)

```
1  declare
2      co_hello_world constant string(30 char) := 'Hello World';
3  begin
4      dbms_output.put_line(co_hello_world);
5  end;
6  /
```

EXAMPLE (GOOD)

```
1  declare
2      co_hello_world constant string(30 char) := 'Hello World';
3  begin
4      sys.dbms_output.put_line(co_hello_world);
5  end;
6  /
```

Object Types

There are no object type-specific recommendations to be defined at the time of writing.

Triggers

G-7710: Avoid cascading triggers.

Major

Maintainability, Testability

REASON

Having triggers that act on other tables in a way that causes triggers on that table to fire lead to obscure behavior.

EXAMPLE (BAD)

```
1  create or replace trigger dept_br_u
2  before update on departments for each row
3  begin
4      insert into departments_hist (department_id
5                                   ,department_name
6                                   ,manager_id
7                                   ,location_id
8                                   ,modification_date)
9      values (:old.department_id
10             ,:old.department_name
11             ,:old.manager_id
12             ,:old.location_id
13             ,sysdate);
14  end;
15  /
16  create or replace trigger dept_hist_br_i
17  before insert on departments_hist for each row
18  begin
19      insert into departments_log (department_id
20                                  ,department_name
21                                  ,modification_date)
22      values (:new.department_id
23              ,:new.department_name
24              ,sysdate);
25  end;
26  /
```

EXAMPLE (GOOD)

```
1  create or replace trigger dept_br_u
2  before update on departments for each row
3  begin
4      insert into departments_hist (department_id
5                                   , department_name
6                                   , manager_id
7                                   , location_id
8                                   , modification_date)
9      values (:old.department_id
10             , :old.department_name
11             , :old.manager_id
12             , :old.location_id
13             , sysdate);
14
15      insert into departments_log (department_id
16                                  , department_name
17                                  , modification_date)
18      values (:old.department_id
19             , :old.department_name
20             , sysdate);
21
22  end;
23  /
```


G-7720: Never use multiple UPDATE OF in trigger event clause.

Blocker

Maintainability, Reliability, Testability

REASON

A DML trigger can have multiple triggering events separated by `or` like `before insert or delete or update of some_column`. If you have multiple `update of` separated by `or`, only one of them (the last one) is actually used and you get no error message, so you have a bug waiting to happen. Instead you always should use a single `update of` with all columns comma-separated, or an `update` without `of` if you wish all columns.

EXAMPLE (BAD)

```
1  create or replace trigger dept_br_u
2  before update of department_id or update of department_name
3  on departments for each row
4  begin
5      -- will only fire on updates of department_name
6      insert into departments_log (department_id
7                                  , department_name
8                                  , modification_date)
9      values (:old.department_id
10             , :old.department_name
11             , sysdate);
12 end;
13 /
```

EXAMPLE (GOOD)

```
1  create or replace trigger dept_br_u
2  before update of department_id, department_name
3  on departments for each row
4  begin
5      insert into departments_log (department_id
6                                  , department_name
7                                  , modification_date)
8      values (:old.department_id
9             , :old.department_name
10            , sysdate);
11 end;
12 /
```

G-7730: Avoid multiple DML events per trigger if primary key is assigned in trigger.

Major

Efficiency, Reliability

REASON

If a trigger makes assignment to the primary key anywhere in the trigger code, that causes the session firing the trigger to take a lock on any child tables with a foreign key to this primary key. Even if the assignment is in for example an `if inserting` block and the trigger is fired by an `update` statement, such locks still happen unnecessarily. The issue is avoided by having one trigger for the insert containing the primary key assignment, and another trigger for the update. Or even better by handling the insert assignment as 'default on null' clauses, so that only an `on update` trigger is needed.

EXAMPLE (BAD)

```
1  create or replace trigger dept_br_iu
2  before insert or update
3  on departments for each row
4  begin
5      if inserting then
6          :new.department_id := department_seq.nextval;
7          :new.created_date := sysdate;
8      end if;
9      if updating then
10         :new.changed_date := sysdate;
11     end if;
12 end;
13 /
```

EXAMPLE (BETTER)

```
1  create or replace trigger dept_br_i
2  before insert
3  on departments for each row
4  begin
5      :new.department_id := department_seq.nextval;
6      :new.created_date := sysdate;
7  end;
8  /
9
10 create or replace trigger dept_br_u
11 before update
12 on departments for each row
13 begin
14     :new.changed_date := sysdate;
15 end;
16 /
```

EXAMPLE (GOOD)

```
1  alter table department modify department_id default on null department_seq.nextval;
2  alter table department modify created_date default on null sysdate;
3
4  create or replace trigger dept_br_u
5  before update
6  on departments for each row
7  begin
8      :new.changed_date := sysdate;
9  end;
10 /
```

Sequences

G-7810: Never use SQL inside PL/SQL to read sequence numbers (or SYSDATE).

Major

Efficiency, Maintainability

REASON

Since ORACLE 11g it is no longer needed to use a `select` statement to read a sequence (which would imply a context switch).

EXAMPLE (BAD)

```
1  declare
2      l_sequence_number employees.employee_id%type;
3  begin
4      select employees_seq.nextval
5          into l_sequence_number
6          from dual;
7  end;
8  /
```

EXAMPLE (GOOD)

```
1  declare
2      l_sequence_number employees.employee_id%type;
3  begin
4      l_sequence_number := employees_seq.nextval;
5  end;
6  /
```

Patterns

Checking the Number of Rows

G-8110: Never use SELECT COUNT(*) if you are only interested in the existence of a row.

Major

Efficiency

REASON

If you do a `select count(*)` all rows will be read according to the `where` clause, even if only the availability of data is of interest. For this we have a big performance overhead. If we do a `select count(*) ... where rownum = 1` there is also a overhead as there will be two communications between the PL/SQL and the SQL engine. See the following example for a better solution.

EXAMPLE (BAD)

```
1  declare
2      l_count pls_integer;
3      co_zero  constant simple_integer := 0;
4      co_salary constant employees.salary%type := 5000;
5  begin
6      select count(*)
7          into l_count
8          from employees
9          where salary < co_salary;
10     if l_count > co_zero then
11         <<emp_loop>>
12         for r_emp in (select employee_id
13                       from employees)
14             loop
15                 if r_emp.salary < co_salary then
16                     my_package.my_proc(in_employee_id => r_emp.employee_id);
17                 end if;
18             end loop emp_loop;
19     end if;
20 end;
21 /
```

EXAMPLE (GOOD)

```
1  declare
2      co_salary constant employees.salary%type := 5000;
3  begin
4      <<emp_loop>>
5      for r_emp in (select e1.employee_id
6                    from employees e1
7                    where exists(select e2.salary
8                                from employees e2
9                                where e2.salary < co_salary))
10         loop
11             my_package.my_proc(in_employee_id => r_emp.employee_id);
12         end loop emp_loop;
13 end;
14 /
```

G-8120: Never check existence of a row to decide whether to create it or not.

Major

Efficiency, Reliability

REASON

The result of an existence check is a snapshot of the current situation. You never know whether in the time between the check and the (insert) action someone else has decided to create a row with the values you checked. Therefore, you should only rely on constraints when it comes to prevention of duplicate records.

EXAMPLE (BAD)

```
1  create or replace package body department_api is
2      procedure ins (in_r_department in departments%rowtype) is
3          l_count pls_integer;
4      begin
5          select count(*)
6              into l_count
7              from departments
8              where department_id = in_r_department.department_id;
9
10         if l_count = 0 then
11             insert into departments
12                 values in_r_department;
13         end if;
14     end ins;
15 end department_api;
16 /
```

EXAMPLE (GOOD)

```
1  create or replace package body department_api is
2      procedure ins (in_r_department in departments%rowtype) is
3      begin
4          insert into departments
5              values in_r_department;
6      exception
7          when dup_val_on_index then null; -- handle exception
8      end ins;
9  end department_api;
10 /
```

Access objects of foreign application schemas

G-8210: Always use synonyms when accessing objects of another application schema.

Major

Changeability, Maintainability

REASON

If a connection is needed to a table that is placed in a foreign schema, using synonyms is a good choice. If there are structural changes to that table (e.g. the table name changes or the table changes into another schema) only the synonym has to be changed no changes to the package are needed (single point of change). If you only have read access for a table inside another schema, or there is another reason that does not allow you to change data in this table, you can switch the synonym to a table in your own schema. This is also good practice for testers working on test systems.

EXAMPLE (BAD)

```
1  declare
2      l_product_name oe.products.product_name%type;
3      co_price constant oe.products@list_price%type := 1000;
4  begin
5      select p.product_name
6          into l_product_name
7          from oe.products p
8          where list_price > co_price;
9  exception
10     when no_data_found then
11         null; -- handle_no_data_found;
12     when too_many_rows then
13         null; -- handle_too_many_rows;
14 end;
15 /
```

EXAMPLE (GOOD)

```
1  create synonym oe_products for oe.products;
2
3  declare
4      l_product_name oe_products.product_name%type;
5      co_price constant oe_products.list_price%type := 1000;
6  begin
7      select p.product_name
8          into l_product_name
9          from oe_products p
10         where list_price > co_price;
11 exception
12     when no_data_found then
13         null; -- handle_no_data_found;
14     when too_many_rows then
15         null; -- handle_too_many_rows;
16 end;
17 /
```

Validating input parameter size

G-8310: Always validate input parameter size by assigning the parameter to a size limited variable in the declaration section of program unit.

Minor

Maintainability, Reliability, Reusability, Testability

REASON

This technique raises an error (`value_error`) which may not be handled in the called program unit. This is the right way to do it, as the error is not within this unit but when calling it, so the caller should handle the error.

EXAMPLE (BAD)

```
1  create or replace package body department_api is
2      function dept_by_name (in_dept_name in departments.department_name%type)
3          return departments%rowtype is
4          l_return departments%rowtype;
5      begin
6          if in_dept_name is null
7             or length(in_dept_name) > 20
8          then
9              raise err.e_param_to_large;
10         end if;
11         -- get the department by name
12         select *
13             from departments
14             where department_name = in_dept_name;
15
16         return l_return;
17     end dept_by_name;
18 end department_api;
19 /
```

EXAMPLE (GOOD)

```
1  create or replace package body department_api is
2      function dept_by_name (in_dept_name in departments.department_name%type)
3          return departments%rowtype is
4          l_dept_name departments.department_name%type not null := in_dept_name;
5          l_return departments%rowtype;
6      begin
7          -- get the department by name
8          select *
9              from departments
10             where department_name = l_dept_name;
11
12         return l_return;
13     end dept_by_name;
14 end department_api;
15 /
```

FUNCTION CALL


```
1  ...
2      r_department := department_api.dept_by_name('Far to long name of a department');
3  ...
4  exception
5      when value_error then ...
```

Ensure single execution at a time of a program unit

G-8410: Always use application locks to ensure a program unit is only running once at a given time.

 **Minor**

Efficiency, Reliability

 **Unsupported in PL/SQL Cop Validators**

We cannot identify where an application lock would make sense. Algorithms to detect wrong, missing and right usages of this pattern are virtually impossible to implement without understanding the context.

REASON

This technique allows us to have locks across transactions as well as a proven way to clean up at the end of the session.

The alternative using a table where a “Lock-Row” is stored has the disadvantage that in case of an error a proper cleanup has to be done to “unlock” the program unit.

EXAMPLE (BAD)

```

1  -- Bad
2  /* Example */
3  create or replace package body lock_up is
4      -- manage locks in a dedicated table created as follows:
5      -- CREATE TABLE app_locks (
6      --     lock_name VARCHAR2(128 CHAR) NOT NULL primary key
7      -- );
8
9      procedure request_lock (in_lock_name in varchar2) is
10     begin
11         -- raises dup_val_on_index
12         insert into app_locks (lock_name) values (in_lock_name);
13     end request_lock;
14
15     procedure release_lock(in_lock_name in varchar2) is
16     begin
17         delete from app_locks where lock_name = in_lock_name;
18     end release_lock;
19 end lock_up;
20 /
21
22 /* Call bad example */
23 declare
24     co_lock_name constant varchar2(30 char) := 'APPLICATION_LOCK';
25 begin
26     lock_up.request_lock(in_lock_name => co_lock_name);
27     -- processing
28     lock_up.release_lock(in_lock_name => co_lock_name);
29 exception
30     when others then
31         -- log error
32         lock_up.release_lock(in_lock_name => co_lock_name);
33         raise;
34 end;
35 /

```

EXAMPLE (GOOD)

```

1  /* Example */
2  create or replace package body lock_up is
3      function request_lock(
4          in_lock_name          in varchar2,
5          in_release_on_commit in boolean := false)
6      return varchar2 is
7          l_lock_handle varchar2(128 char);
8      begin
9          sys.dbms_lock.allocate_unique(
10             lockname      => in_lock_name,
11             lockhandle     => l_lock_handle,
12             expiration_secs => constants_up.co_one_week
13         );
14         if sys.dbms_lock.request(
15             lockhandle     => l_lock_handle,
16             lockmode       => sys.dbms_lock.x_mode,
17             timeout        => sys.dbms_lock.maxwait,
18             release_on_commit => coalesce(in_release_on_commit, false)
19         ) > 0
20         then
21             raise err.e_lock_request_failed;
22         end if;
23         return l_lock_handle;
24     end request_lock;
25
26     procedure release_lock(in_lock_handle in varchar2) is
27     begin
28         if sys.dbms_lock.release(lockhandle => in_lock_handle) > 0 then
29             raise err.e_lock_request_failed;
30         end if;
31     end release_lock;
32 end lock_up;
33 /
34
35 /* Call good example */
36 declare
37     l_handle varchar2(128 char);
38     co_lock_name constant varchar2(30 char) := 'APPLICATION_LOCK';
39 begin
40     l_handle := lock_up.request_lock(in_lock_name => co_lock_name);
41     -- processing
42     lock_up.release_lock(in_lock_handle => l_handle);
43 exception
44     when others then
45         -- log error
46         lock_up.release_lock(in_lock_handle => l_handle);
47         raise;
48 end;
49 /

```

Use dbms_application_info package to follow progress of a process

G-8510: Always use dbms_application_info to track program process transiently.

Minor

Efficiency, Reliability

Unsupported in PL/SQL Cop Validators

We cannot know where the use of `dbms_application_info` is sensible. Algorithms to detect wrong, missing and right usages of this pattern are virtually impossible to implement without understanding the context.

REASON

This technique allows us to view progress of a process without having to persistently write log data in either a table or a file. The information is accessible through the `v$sqlsession` view.

EXAMPLE (BAD)

```
1  create or replace package body employee_api is
2      procedure process_emps is
3      begin
4          <<employees>>
5          for emp_rec in (select employee_id
6                          from employees
7                          order by employee_id)
8          loop
9              null; -- some processing
10         end loop employees;
11     end process_emps;
12 end employee_api;
13 /
```

EXAMPLE (GOOD)

```
1  create or replace package body employee_api is
2      procedure process_emps is
3      begin
4          sys.dbms_application_info.set_module(module_name => $$plsql_unit
5                                              , action_name => 'Init');
6          <<employees>>
7          for emp_rec in (select employee_id
8                          from employees
9                          order by employee_id)
10         loop
11             sys.dbms_application_info.set_action('Processing ' || emp_rec.employee_id);
12         end loop employees;
13     end process_emps;
14 end employee_api;
15 /
```

Function Usage

G-9010: Always use a format model in string to date/time conversion functions.

Major

Changeability, Maintainability, Reliability, Security, Testability

Reason

Converting from strings to `date` or `timestamp` datatypes (using `to_date`, `to_timestamp`, `to_timestamp_tz` or `cast` to any of those datatypes) in practice always expects a fixed format (unlike converting to strings that can be fixed as well as allow the session to decide). Therefore it is a bad idea to allow this conversion to rely on the session NLS settings (`nls_date_format`, `nls_timestamp_format` and `nls_timestamp_tz_format`) as this makes the code vulnerable to changes in session and/or server configuration. It is even possible to utilize session `nls_date_format` for SQL injection if you use dynamic SQL.

Using an explicit format model for string to `date` or `timestamp` conversion avoids this inappropriate dependability on configurable NLS parameters.

Example (bad)

```
1  create package body employee_api is
2      procedure set_dob (in_employee_id in employees.employee_id%type, in_dob_str in varchar2)
3      begin
4          update employee
5              set date_of_birth = to_date(in_dob_str)
6              where employee_id = in_employee_id;
7      end set_dob;
8  end employee_api;
9  /
```

Example (good)

```
1  create package body employee_api is
2      procedure set_dob (in_employee_id in employees.employee_id%type, in_dob_str in varchar2)
3      begin
4          update employee
5              set date_of_birth = to_date(in_dob_str, 'YYYY-MM-DD')
6              where employee_id = in_employee_id;
7      end set_dob;
8  end employee_api;
9  /
```

G-9020: Try to use a format model and NLS_NUMERIC_CHARACTERS in string to number conversion functions.

Major

Changeability, Maintainability, Reliability, Security, Testability

Reason

Converting from strings to numeric datatypes (using `to_number`, `to_binary_double`, `to_binary_float` or `cast` to any of those datatypes) rely on session NLS settings for `nls_numeric_characters`. Typically the input string is expected to have a given decimal- and group-separator, so it is best practice to specify `nls_numeric_characters` in the function call. However, this requires also setting a format model, which is a good idea but can require a very large format model with many 9's if you do not know the maximum length of the string.

To avoid an inappropriate dependability on configurable NLS parameters, try to use both format model and `nls_numeric_characters` in the conversion function call. The exceptions can be if the input is known to always be integer with no decimal- or group-separator, or if you do not know the maximum number of digits **and** have control over the session `nls_numeric_characters` parameter.

Example (bad)

```
1  create package body employee_api is
2      procedure set_salary (in_employee_id in employees.employee_id%type, in_salary in varchar2)
3      begin
4          update employee
5          set salary = to_number(in_salary)
6          where employee_id = in_employee_id;
7      end set_dob;
8  end employee_api;
9  /
```

Example (good)

```
1  create package body employee_api is
2      procedure set_salary (in_employee_id in employees.employee_id%type, in_salary in varchar2)
3      begin
4          update employee
5          set salary = to_number(in_salary, '99999999999999999999.99999',
6          q'[nls_numeric_characters='.',']')
7          where employee_id = in_employee_id;
8      end set_dob;
9  end employee_api;
10 /
```

G-9030: Try to define a default value on conversion errors.

Minor

Maintainability, Reliability, Testability

Reason

When converting from strings to other datatypes using a conversion function that supports the `default ... on conversion error` clause, it is a good idea to use this clause to avoid getting an error raised on bad input. The exception can be when you explicitly *want* an error to be raised to catch and process it in a later exception handler.

Example (bad)

```
1  create package body employee_api is
2      procedure set_dob (in_employee_id in employees.employee_id%type, in_dob_str in varchar2)
3      begin
4          update employee
5              set date_of_birth = to_date(in_dob_str, 'YYYY-MM-DD')
6              where employee_id = in_employee_id;
7      end set_dob;
8  end employee_api;
9  /
```

Example (good)

```
1  create package body employee_api is
2      procedure set_dob (in_employee_id in employees.employee_id%type, in_dob_str in varchar2)
3      begin
4          update employee
5              set date_of_birth = to_date(in_dob_str default null on conversion error, 'YYYY-MM-DD')
6              where employee_id = in_employee_id;
7      end set_dob;
8  end employee_api;
9  /
```


Complexity Analysis

Using software metrics like complexity analysis will guide you towards maintainable and testable pieces of code by reducing the complexity and splitting the code into smaller chunks.

Halstead Metrics

Calculation

First, we need to compute the following numbers, given the program:

- n_1 = the number of distinct operators
- n_2 = the number of distinct operands
- N_1 = the total number of operators
- N_2 = the total number of operands

From these numbers, five measures can be calculated:

- Program length:

$$N = N_1 + N_2$$

- Program vocabulary:

$$n = n_1 + n_2$$

- Volume:

$$V = N \cdot \log_2 n$$

- Difficulty:

$$D = \frac{n_1}{2} \cdot \frac{N_2}{n_2}$$

- Effort:

$$E = D \cdot V$$

The difficulty measure

D is related to the difficulty of the program to write or understand, e.g. when doing code review.

The volume measure

V describes the size of the implementation of an algorithm.

McCabe's Cyclomatic Complexity

Description

Cyclomatic complexity (or conditional complexity) is a software metric used to measure the complexity of a program. It directly measures the number of linearly independent paths through a program's source code.

Cyclomatic complexity is computed using the control flow graph of the program: the nodes of the graph correspond to indivisible groups of commands of a program, and a directed edge connects two nodes if the second command might be executed immediately after the first command. Cyclomatic complexity may also be applied to individual functions, modules, methods or classes within a program.

The cyclomatic complexity of a section of source code is the count of the number of linearly independent paths through the source code. For instance, if the source code contains no decision points, such as `if` statements or `for` loops, the complexity would be 1, since there is only a single path through the code. If the code has a single `if` statement containing a single condition there would be two paths through the code, one path where the `if` statement is evaluated as `true` and one path where the `if` statement is evaluated as `false`.

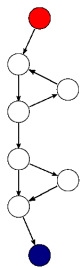
Calculation

Mathematically, the cyclomatic complexity of a structured program is defined with reference to a directed graph containing the basic blocks of the program, with an edge between two basic blocks if control may pass from the first to the second (the control flow graph of the program). The complexity is then defined as:

$$M = E - N + 2P$$

where

- M = cyclomatic complexity
- E = the number of edges of the graph
- N = the number of nodes of the graph
- P = the number of connected components.



Take, for example, a control flow graph of a simple program. The program begins executing at the red node, then enters a loop (group of three nodes immediately below the red node). On exiting the loop, there is a conditional statement (group below the loop), and finally the program exits at the blue node. For this graph,

$E = 9$,

$N = 8$ and

$P = 1$, so the cyclomatic complexity of the program is

3.

```

1  begin
2      for i in 1..3
3          loop
4              dbms_output.put_line('in loop');
5          end loop;
6          --
7          if 1 = 1
8              then
9                  dbms_output.put_line('yes');
10             end if;
11             --
12             dbms_output.put_line('end');
13         end;
14     /

```

For a single program (or subroutine or method), P is always equal to 1. Cyclomatic complexity may, however, be applied to several such programs or subprograms at the same time (e.g., to all of the methods in a class), and in these cases P will be equal to the number of programs in question, as each subprogram will appear as a disconnected subset of the graph.

It can be shown that the cyclomatic complexity of any structured program with only one entrance point and one exit point is equal to the number of decision points (i.e., `if` statements or conditional loops) contained in that program plus

one.

Cyclomatic complexity may be extended to a program with multiple exit points; in this case it is equal to:

$$\pi = s + 2$$

Where

- π is the number of decision points in the program, and
- s is the number of exit points.

Code Reviews

Code reviews check the results of software engineering. According to IEEE-Norm 729, a review is a more or less planned and structured analysis and evaluation process. Here we distinguish between code review and architect review.

To perform a code review means that after or during the development one or more reviewer proof-reads the code to find potential errors, potential areas for simplification, or test cases. A code review is a very good opportunity to save costs by fixing issues before the testing phase.

What can a code-review be good for?

- Code quality
- Code clarity and maintainability
- Quality of the overall architecture
- Quality of the documentation
- Quality of the interface specification

For an effective review, the following factors must be considered:

- Definition of clear goals.
- Choice of a suitable person with constructive critical faculties.
- Psychological aspects.
- Selection of the right review techniques.
- Support of the review process from the management.
- Existence of a culture of learning and process optimization.

Requirements for the reviewer:

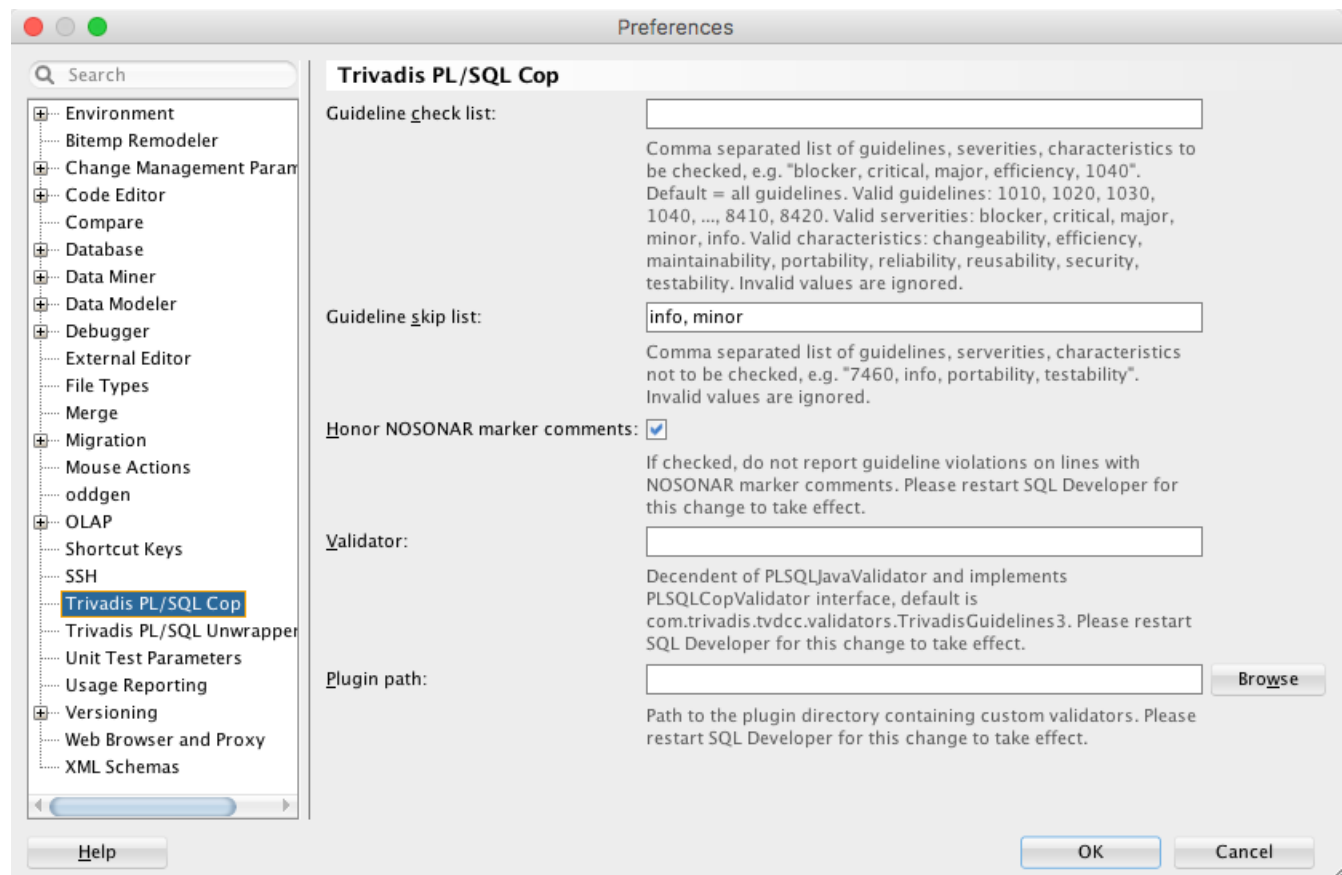
- He must not be the owner of the code.
- Code reviews may be unpleasant for the developer, as he could fear that his code will be criticized. If the critic is not considerate, the code writer will build up rejection and resistance against code reviews.

Tool Support

Development

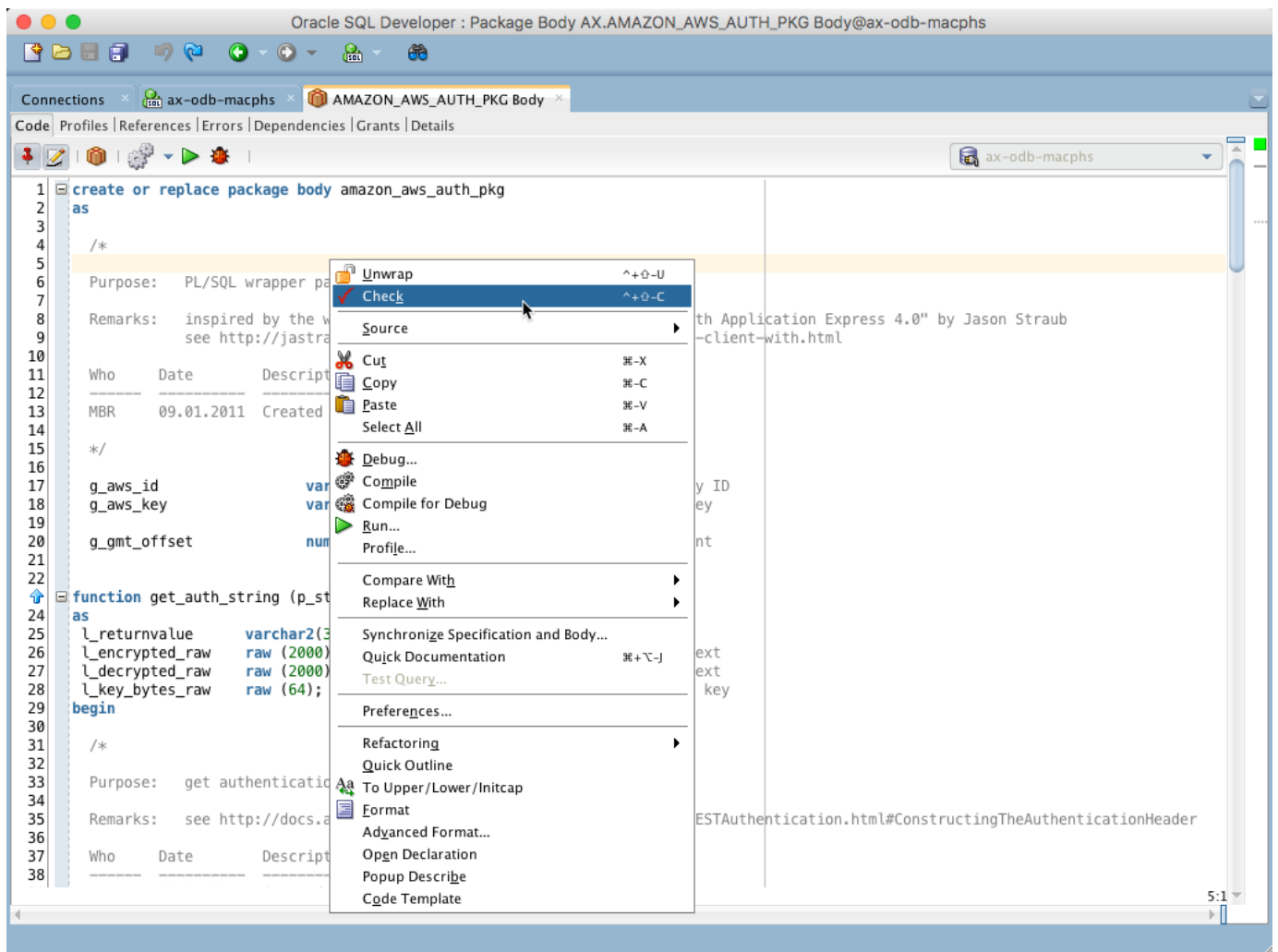
Trivadis offers a cost-free extension to ORACLE SQL Developer to test compliance with this coding guideline. The extension may be parameterized to your preferred set of rules and allows checking this set against a program unit.

Setting the preferences



There is an include list as well as an exclude list to define which rules to be checked or ignored.

Activate PLSQL Cop using context menu



The result of the checking process is a list of violations with direct links to the place in the code as well as software metrics like:

- Cyclomatic complexity
- Halstead volume
- Maintainability Index
- Number of lines of code
- Number of comment lines
- Issue Overview

This statistics are gathered for each program unit in the reviewed code.

Software metrics

Oracle SQL Developer

Trivadis PL/SQL Cop

IssuesReport

Trivadis PL/SQL Cop Version 2.0.0.2244 for Trivadis PL/SQL & SQL Coding Guidelines Version 3.2

Copyright 2010–2017 by Trivadis AG
Sägereistrasse 29
CH-8152 Glattbrugg (Zurich)
www.trivadis.com

Parameters

check (all guidelines)
skip info, minor
nosonar yes – honor NOSONAR marker comments (do not report guideline violations on lines with NOSONAR marker comments)
validator (default validator – com.trivadis.tvdcc.validators.TrivadisGuidelines3)
plugin-path (no directory configured)

SQL Developer Editor

Title Package Body AX.AMAZON_AWS_AUTH_PKG@ax-odb-macphs
Date/time 2017-01-29 18:21:16

Metrics

Number of bytes 4,962
Number of lines (LOC) 246
Number of comment lines 111
Number of blank lines 58
Number of net lines 77
Number of commands 1
Number of statements (PL/SQL) 22
Max. cyclomatic complexity 2 (< 11 11..50 > 50)
Max. Halstead volume 333 (< 1001 1001..3000 > 3000)
Min. maintainability index (MI) 141 (> 84 64..84 < 64)
Avg cyclomatic complexity 1 (< 11 11..50 > 50)
Avg Halstead volume 77 (< 1001 1001..3000 > 3000)
Avg maintainability index (MI) 161 (> 84 64..84 < 64)
Number of issues 11
Number of warnings 11
Number of errors 0

PL/SQL Units

PL/SQL Unit	Line	# Lines	# Comment lines	# Blank lines	# Net lines	# Stmts	Cyclomatic complexity	Halstead volume	Maintainability index
amazon_aws_auth_pkg.get_date_string	105	25	11	4	10	6	2	261	142
amazon_aws_auth_pkg.get_auth_string	29	26	12	7	8	6	1	333	141
amazon_aws_auth_pkg.init	224	19	11	3	5	3	1	39	164
amazon_aws_auth_pkg.get_epoch	135	19	11	4	4	2	1	94	160
amazon_aws_auth_pkg.get_signature	60	17	11	3	3	1	1	30	171
amazon_aws_auth_pkg.get_aws_id	81	17	11	3	3	1	1	6	180
amazon_aws_auth_pkg.set_aws_id	158	18	11	4	3	1	1	10	174
amazon_aws_auth_pkg.set_aws_key	180	17	11	3	3	1	1	10	177
amazon_aws_auth_pkg.set_gmt_offset	201	17	11	3	3	1	1	10	177

Issue Overview

#	%	Severity	Characteristics	Message
6	54.5%	Major	Security	G-7510: Always prefix ORACLE supplied packages with owner schema name.
5	45.5%	Major	Efficiency	G-7460: Try to define your packaged/standalone function to be deterministic if appropriate.

Open

PL/SQL & SQL Coding Guidelines Version 4.0

Page 167 of 177

Appendix

A - PL/SQL & SQL Coding Guidelines as PDF

These guidelines are primarily produced in **HTML** using **Material for MkDocs**.

However, we provide these guidelines also as **PDF** produced by **wkhtmltopdf**.



The formatting is not perfect, but it should be adequate for those who want to work with offline documents.

B - Mapping new guidelines to prior versions

Old Id	New Id	Text	Severity	Change-ability	Efficiency	Maintain-ability	Po-ability
1	1010	Try to label your sub blocks.	Minor			X	
2	1020	Always have a matching loop or block label.	Minor			X	
3	1030	Avoid defining variables that are not used.	Minor		X	X	
4	1040	Avoid dead code.	Minor			X	
5	1050	Avoid using literals in your code.	Minor	X			
6	1060	Avoid storing ROWIDs or UROWIDs in database tables.	Major				
7	1070	Avoid nesting comment blocks.	Minor			X	

n/a	1080	Avoid using the same expression on both sides of a relational comparison operator or a logical operator.	Minor		X	X	
8	2110	Try to use anchored declarations for variables, constants and types.	Major			X	
9	2120	Try to have a single location to define your types.	Minor	X			
10	2130	Try to use subtypes for constructs used often in your code.	Minor	X			
n/a	2135	Avoid assigning values to local variables that are not used by a subsequent statement.	Major		X	X	
11	2140	Never initialize variables with NULL.	Minor			X	
n/a	2145	Never self-assign a variable.	Minor			X	
12	2150	Avoid comparisons with NULL value, consider using IS [NOT] NULL.	Blocker				)
13	2160	Avoid initializing variables using functions in the declaration section.	Critical				
14	2170	Never overload variables.	Major				
15	2180	Never use quoted identifiers.	Major			X	
16	2185	Avoid using overly short names for explicitly or implicitly declared identifiers.	Minor			X	
17	2190	Avoid using ROWID or UROWID.	Major				)
18	2210	Avoid declaring NUMBER variables, constants or subtypes with no precision.	Minor		X		

19	2220	Try to use PLS_INTEGER instead of NUMBER for arithmetic operations with integer values.	Minor		X		
n/a	2230	Try to use SIMPLE_INTEGER datatype when appropriate.	Minor		X		
20	2310	Avoid using CHAR data type.	Major				
21	2320	Never use VARCHAR data type.	Major				)
22	2330	Never use zero-length strings to substitute NULL.	Major				)
23	2340	Always define your VARCHAR2 variables using CHAR SEMANTIC (if not defined anchored).	Minor				
24	2410	Try to use boolean data type for values with dual meaning.	Minor			X	
25	2510	Avoid using the LONG and LONG RAW data types.	Major				)
n/a	2610	Never use self-defined weak ref cursor types.	Minor	X		X	)
26	3110	Always specify the target columns when coding an insert statement.	Major			X	
n/a	3115	Avoid self-assigning a column.	Minor			X	
27	3120	Always use table aliases when your SQL statement involves more than one source.	Major			X	
28	3130	Try to use ANSI SQL-92 join syntax.	Minor			X	)
29	3140	Try to use anchored records as targets for your cursors.	Major			X	
n/a	3150	Try to use identity columns for surrogate keys.	Minor			X	
n/a	3160	Avoid visible virtual columns.	Major			X	

n/a	3170	Always use DEFAULT ON NULL declarations to assign default values to table columns if you refuse to store NULL values.	Major				
n/a	3180	Always specify column names instead of positional references in ORDER BY clauses.	Major	X			
n/a	3185	Never use ROWNUM at the same query level as ORDER BY.	Major				
n/a	3190	Avoid using NATURAL JOIN.	Major	X			
n/a	3195	Always use wildcards in a LIKE clause.	Minor			X	
30	3210	Always use BULK OPERATIONS (BULK COLLECT, FORALL) whenever you have to execute a DML statement for more than 4 times.	Major		X		
n/a	3310	Never commit within a cursor loop.	Critical		X		
n/a	3320	Try to move transactions within a non-cursor loop into procedures.	Major			X	
31	4110	Always use %NOTFOUND instead of NOT %FOUND to check whether a cursor returned data.	Minor			X	
32	4120	Avoid using %NOTFOUND directly after the FETCH when working with BULK OPERATIONS and LIMIT clause.	Critical				
33	4130	Always close locally opened cursors.	Major		X		
34	4140	Avoid executing any statements between a SQL operation and the usage of an implicit cursor attribute.	Major				

35	4210	Try to use CASE rather than an IF statement with multiple ELSIF paths.	Major			X	
36	4220	Try to use CASE rather than DECODE.	Minor			X	
37	4230	Always use a COALESCE instead of a NVL command, if parameter 2 of the NVL function is a function call or a SELECT statement.	Critical		X		
38	4240	Always use a CASE instead of a NVL2 command if parameter 2 or 3 of NVL2 is either a function call or a SELECT statement.	Critical		X		
n/a	4250	Avoid using identical conditions in different branches of the same IF or CASE statement.	Major			X	
n/a	4260	Avoid inverting boolean conditions with NOT.	Minor			X	
n/a	4270	Avoid comparing boolean values to boolean literals.	Minor			X	
39	4310	Never use GOTO statements in your code.	Major			X	
40	4320	Always label your loops.	Minor			X	
n/a	4325	Never reuse labels in inner scopes.	Major			X	
41	4330	Always use a CURSOR FOR loop to process the complete cursor results unless you are using bulk operations.	Minor			X	
42	4340	Always use a NUMERIC FOR loop to process a dense array.	Minor			X	
43	4350	Always use 1 as lower and COUNT() as upper bound when looping through a dense array.	Major				

44	4360	Always use a WHILE loop to process a loose array.	Minor		X		
45	4370	Avoid using EXIT to stop loop processing unless you are in a basic loop.	Major			X	
46	4375	Always use EXIT WHEN instead of an IF statement to exit from a loop.	Minor			X	
47	4380	Try to label your EXIT WHEN statements.	Minor			X	
48	4385	Never use a cursor for loop to check whether a cursor returns data.	Major		X		
49	4390	Avoid use of unreferenced FOR loop indexes.	Major		X		
50	4395	Avoid hard-coded upper or lower bound values with FOR loops.	Minor	X		X	
n/a	5010	Try to use a error/logging framework for your application.	Critical				
51	5020	Never handle unnamed exceptions using the error number.	Critical			X	
52	5030	Never assign predefined exception names to user defined exceptions.	Blocker				
53	5040	Avoid use of WHEN OTHERS clause in an exception section without any other specific handlers.	Major				
54	n/a	Avoid use of EXCEPTION_INIT pragma for a 20nnn error.	Major				
55	5050	Avoid use of the RAISE_APPLICATION_ERROR built-in procedure with a hard-coded 20nnn error number or hard-coded message.	Major	X		X	

56	5060	Avoid unhandled exceptions.	Major				
57	5070	Avoid using Oracle predefined exceptions.	Critical				
n/a	5080	Always use FORMAT_ERROR_BACKTRACE when using FORMAT_ERROR_STACK or SQLERRM.	Minor			X	
58	6010	Always use a character variable to execute dynamic SQL.	Major			X	
59	6020	Try to use output bind arguments in the RETURNING INTO clause of dynamic DML statements rather than the USING clause.	Minor			X	
60	7110	Try to use named notation when calling program units.	Major	X		X	
61	7120	Always add the name of the program unit to its end keyword.	Minor			X	
n/a	7125	Always use CREATE OR REPLACE instead of CREATE alone.	Minor			X	
62	7130	Always use parameters or pull in definitions rather than referencing external variables in a local program unit.	Major			X	
63	7140	Always ensure that locally defined procedures or functions are referenced.	Major			X	
64	7150	Try to remove unused parameters.	Minor		X	X	
n/a	7160	Always explicitly state parameter mode.	Major			X	
n/a	7170	Avoid using an IN OUT parameter as IN or OUT only.	Major		X	X	

65	7210	Try to keep your packages small. Include only few procedures and functions that are used in the same context.	Minor		X	X	
66	7220	Always use forward declaration for private functions and procedures.	Minor	X			
67	7230	Avoid declaring global variables public.	Major				
n/a	7250	Never use RETURN in package initialization block.	Minor			X	
69	7310	Avoid standalone procedures – put your procedures in packages.	Minor			X	
70	7320	Avoid using RETURN statements in a PROCEDURE.	Major			X	
n/a	7330	Always assign values to OUT parameters.	Major			X	
71	7410	Avoid standalone functions – put your functions in packages.	Minor			X	
73	7420	Always make the RETURN statement the last statement of your function.	Major			X	
72	7430	Try to use no more than one RETURN statement within a function.	Major			X	
74	7440	Never use OUT parameters to return values from a function.	Major				
75	7450	Never return a NULL value from a BOOLEAN function.	Major				
n/a	7460	Try to define your packaged/standalone function deterministic if appropriate.	Major		X		
76	7510	Always prefix ORACLE supplied packages with owner schema name.	Major				

77	7710	Avoid cascading triggers.	Major			X	
n/a	7720	Never use multiple UPDATE OF in trigger event clause.	Blocker			X	
n/a	7730	Avoid multiple DML events per trigger if primary key is assigned in trigger.	Major		X		
n/a	7810	Never use SQL inside PL/SQL to read sequence numbers (or SYSDATE).	Major		X	X	
78	8110	Never use SELECT COUNT(*) if you are only interested in the existence of a row.	Major		X		
n/a	8120	Never check existence of a row to decide whether to create it or not.	Major		X		
79	8210	Always use synonyms when accessing objects of another application schema.	Major	X		X	
n/a	8310	Always validate input parameter size by assigning the parameter to a size limited variable in the declaration section of program unit.	Minor			X	
n/a	8410	Always use application locks to ensure a program unit is only running once at a given time.	Minor		X		
n/a	8510	Always use dbms_application_info to track program process transiently.	Minor		X		
n/a	9010	Always use a format model in string to date/time conversion functions.	Major	X		X	
n/a	9020	Try to use a format model and NLS_NUMERIC_CHARACTERS in string to number conversion functions.	Major	X		X	
n/a	9030	Try to define a default value on conversion errors.	Minor			X	

1. We see a table and a view as a collection. A jar containing beans is labeled "beans". In Java we call such a collection also "beans" (`List<Bean> beans`) and name an entry "bean" (`for (Bean bean : beans) {...}`). An entry of a table is a row (singular) and a table can contain an unbounded number of rows (plural). This and the fact that the Oracle database uses the same concept for their tables and views lead to the decision to use the plural to name a table or a view. 
2. It used to be good practice to use uppercase keywords and lowercase names to help visualize code structure. But practically all editors support more or less advanced color highlighting of code, similar to the examples in these guidelines. Hence as of version 4.0 we are now recommending all lowercase, as this is easier and faster for the brain to process. You may choose to prefer the old rule - however, it is important to always be consistent, like for example keywords always in uppercase and names always in lowercase. 
3. Tabs are not used because the indentation depends on the editor configuration. We want to ensure that the code looks the same, independent of the editor used. Hence, no tabs. But why not use 8 spaces? That's the traditional value for a tab. When writing a package function the code in the body has an indentation of 3. That's 24 characters as a starting point for the code. We think it's too much. Especially if we try to keep a line below 100 or 80 characters. Other good options would be 2 or 4 spaces. We settled for 3 spaces as a compromise. The indentation is still good visible, but does not use too much space. 